Linguagens Formais, Autômatos e Gramáticas

Profa. Gina M. B. de Oliveira

Universidade Federal de Uberlândia

3 – Linguagens Regulares

Esse material é baseado nos seguintes materiais:

• MENEZES. P. F. B. Linguagens Formais e Autômatos. Editora Sagra Luzzato, Porto Alegre, 4a ed., 2001.

• HOPCROFT, J. E.; ULLMAN, J. D. **Introduction to automata theory languages and computation.** Massachusetts: Addison-Wesley, 1979.

• Transparências de aula do Prof. Paulo F. B. Menezes.

• Transparências de aula da Profa. Gina M. B. de Oliveira

Cap. 3 Linguagens Regulares, Autômatos Finitos e Gramáticas Regulares



Linguagens Regulares ou Tipo 3

- estudos e aplicações são variados e abrangentes
- exemplos
  - editores de texto
  - processadores de texto em geral
  - pesquisa de dados
  - interface simples homem×máquina
  - protocolos de comunicação
  - lógica
  - ...

Linguagens Regulares ou Tipo 3

- formalismos operacionais ou reconhecedores
  - *Autômato Finito Determinístico*
  - *Autômato Finito Não-Determinístico*
  - *Autômato Finito com Movimentos Vazio*
- formalismo axiomático ou gerador
  - *Gramática Regular*
- formalismo denotacional
  - *Expressão Regular*
  - também considerado formalismo gerador

Definição: Sistema de Estados Finitos

•É um modelo matemático de sistema com entradas e saídas discretas;

•Pode assumir um número finito e pré definido de estados;

•Cada estado resume somente as informações do passado necessárias para determinar as ações da próxima entrada.

- ***Exemplo:* Elevador**
  - *entrada*
    - requisições pendentes
  - *estado*
    - andar corrente
    - direção de movimento
  - *não* memoriza as requisições anteriores

Definição: Autômato Finito (Determinístico) – Def. 1

É uma máquina reconhecedora composta de:

Fita de Entrada

Unidade de Controle

Programa ou Função de Transição de Estados

Observação: o Autômato Finito não possui memória auxiliar. As informações do passado são armazenadas através dos estados.

Fita de Entrada:

finita e dividida em células, sendo que cada célula armazena um símbolo. Não é possível gravar.

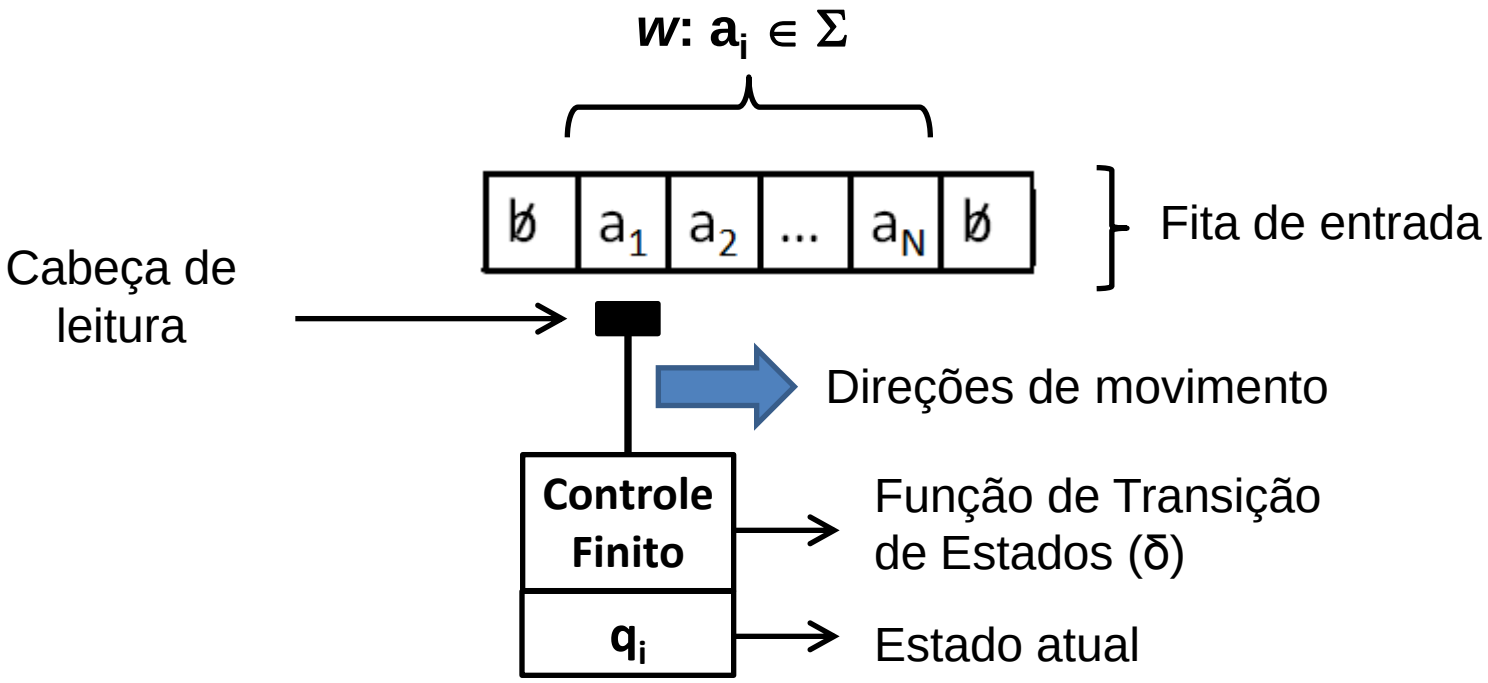
Unidade de Controle:

armazena o estado corrente (dentre um número finito de estados);
possui cabeça de leitura (não grava) que acessa cada célula da fita, lê o símbolo e movimenta-se exclusivamente à DIREITA;
no início a cabeça é posicionada no primeiro símbolo.

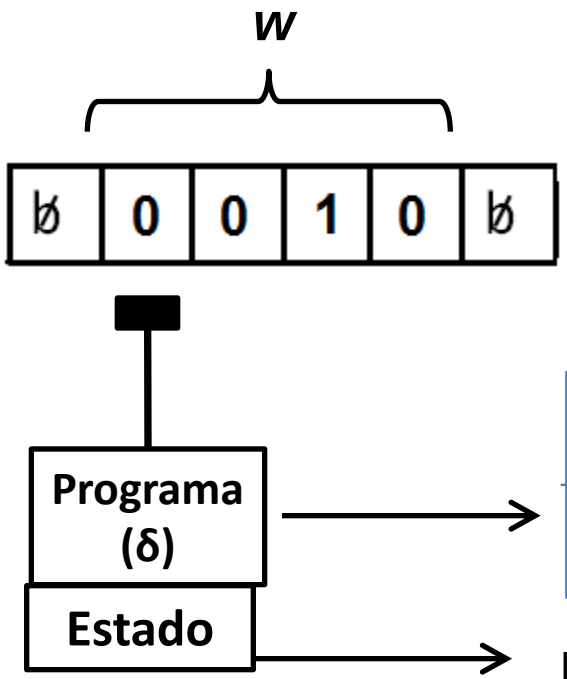
Programa ou Função de Transição de Estados:

função que comanda a leitura e define o estado corrente;
função parcial que define o novo estado do autômato a partir do estado corrente e do símbolo lido na fita.

Esquematicamente:



Exemplo:



Se estado = q0 e símbolo lido = 0: permaneça em q0
Se estado = q0 e símbolo lido = 1: mude para q1
Se estado = q1 e símbolo lido = 0: permaneça em q1
Se estado = q1 e símbolo lido = 1: permaneça em q1

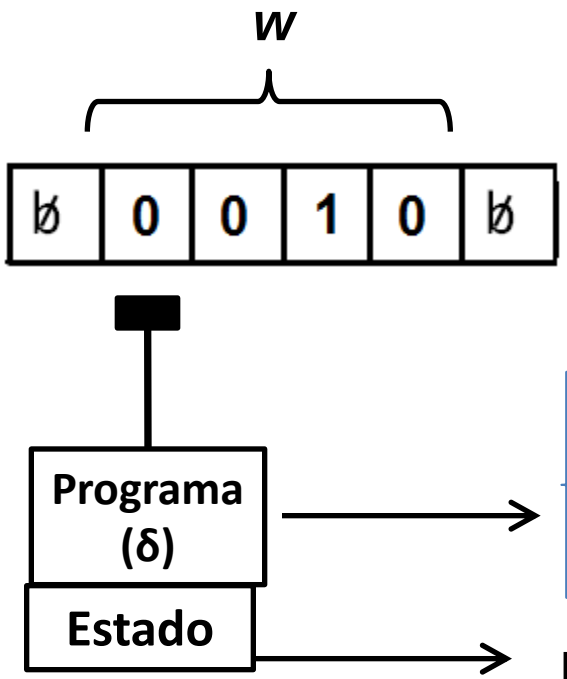
Estado inicial = q0 e Estado final (aceitação) = q1

Que tipo de palavra é reconhecida/aceita por esse AF?

Processamento da palavra 0010:

Processamento da palavra 000:

Exemplo:



Se estado = q0 e símbolo lido = 0: permaneça em q0
Se estado = q0 e símbolo lido = 1: mude para q1
Se estado = q1 e símbolo lido = 0: permaneça em q1
Se estado = q1 e símbolo lido = 1: permaneça em q1

Estado inicial = q0 e Estado final (aceitação) = q1

Que tipo de palavra é reconhecida/aceita por esse AF?
Palavras binárias que tenham pelo menos uma ocorrência de um “1”.

Definição: Autômato Finito (Determinístico) – Def. 2

Um Autômato Finito é uma 5-upla:

$A = (Q, \Sigma, \delta, q_0, F)$

$Q \rightarrow$ conjunto finito de estados

$\Sigma \rightarrow$ alfabeto (símbolos da fita): $\Sigma \cap Q = \varnothing$

$q_0 \rightarrow$ estado inicial

$\delta \rightarrow$ função de transição de estado: $Q \times \Sigma \rightarrow Q$

$F \rightarrow$ conjuntos de estados finais (aceitação): $F \subseteq Q$

Definição: Função de Transição de Estados ou Programa (δ)

Para todos os estados possíveis do autômato (q ϵ Q), a função deve definir qual será o próximo estado do autômato (q' ϵ Q), quando for lido qualquer símbolo na fita (a ϵ $\sum$).

A função δ é dada por uma lista de transições do tipo:

$$\delta(q, a) = q',$$

ou seja,

O autômato finito estando no estado q e lendo o símbolo a na fita de entrada, move a cabeça para a direita e vai para o estado q'.

No exemplo anterior:

$A = (Q, \Sigma, \delta, q_0, F)$

$Q = \{q0, q1\}$

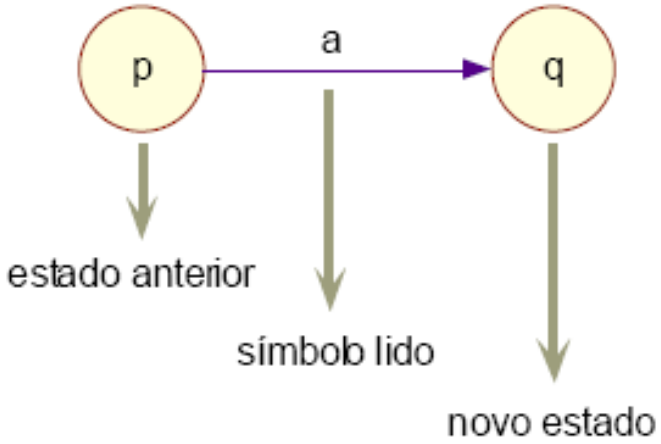
$\Sigma = \{0,1\}$

$F = \{q1\}$

δ: $\delta(q0,0)=q0$, $\delta(q0,1)=q1$, $\delta(q1,0)=q1$, $\delta(q1,1)=q1$

Função δ: Representação em Grafo

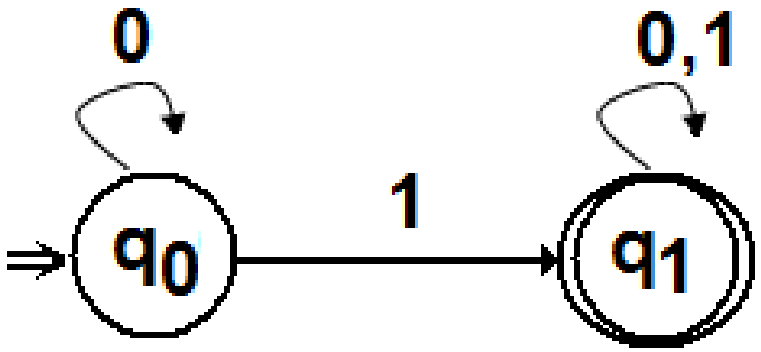
- pode ser interpretada como um grafo finito direto



- representação dos estados
  - inicial
  - finais



No exemplo anterior:



Função δ: Representação em Tabela

	δ	a_0	a_1	⋯	a_N
⇒	q_0	q	q		q
	q_1	q	q		q
	q_2	q	q		q
	⋮				
*	q_f	q	q		q

No exemplo anterior:

	δ	0	1
⇒	q_0	q_0	q_1
*	q_1	q_1	q_1

♦ Processamento

- sucessiva aplicação da função programa
  - para cada símbolo da entrada
  - da esquerda para a direita
  - até parar
- definição formal do comportamento
  - necessário estender a função programa
- argumento da função programa estendida
  - um estado
  - uma *palavra*

Definição:
Função de Transição de Estados Estendida($\underline{\delta}$)

A função $\underline{\delta}$ é uma função $\underline{\delta}$: Q x $\sum^* \rightarrow$ Q.
É a função δ estendida para palavras e é definida por:

$\underline{\delta}(q, \varepsilon) = q$

$\underline{\delta}(q, aw) = \underline{\delta}(\delta(q, a), w)$, sendo w ϵ $\sum^*$, α ϵ .Σ

Para uma palavras w ϵ $\sum^*$: $\underline{\delta}(q, w) = q'$.

O Autômato Finito no estado q após ler todos os símbolos e w (da esquerda para a direita) vai para o estado q'.

Definição: Palavra aceita por um Autômato Finito

Seja A = (Q, $\sum$, δ, q_0, F) e w ϵ $\sum$*. A palavra w é aceita por A se **δ(q0, w) ϵ F**. Se o Autômato Finito A, partindo do estado inicial q_0, após ler todos os símbolos de w, for para um estado final q (q ϵ F), então a palavra w é aceita por A.

Condições de Parada de um Autômato Finito

Um Autômato Finito sempre pára ao processar qualquer entrada, aceitando ou rejeitando w.

W é aceita: se após processar o último símbolo, o AF, para em um estado final.

W é rejeitada: se após processar o último símbolo o AF pára em um estado não final ou a função δ é indefinida para o argumento (estado, símbolo).

Definição: Linguagem Reconhecida (ou aceita) por um Autômato Finito

Seja $A = (Q, \sum, \delta, q_0, F)$ um autômato finito. A Linguagem reconhecida por A é dada por:

$$L(A) = \{w \in \sum^* \mid \underline{\delta}(q_0, w) \in F\}$$

L(A) é o conjunto de todas as palavras pertencentes a $\sum^*$ e aceitas por A.

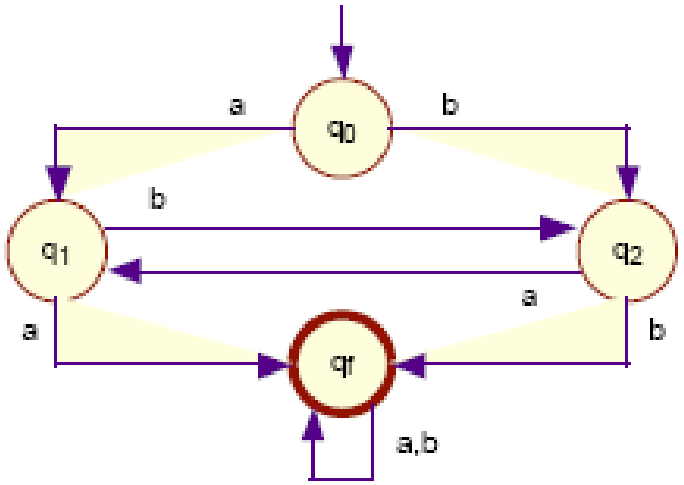
Exemplo:

$M_1 = (\{a, b\}, \{q_0, q_1, q_2, q_f\}, \delta_1, q_0, \{q_f\})$

δ_1	a	b
→ q_0	q_1	q_2
q_1	q_f	q_2
q_2	q_1	q_f
★ q_f	q_f	q_f

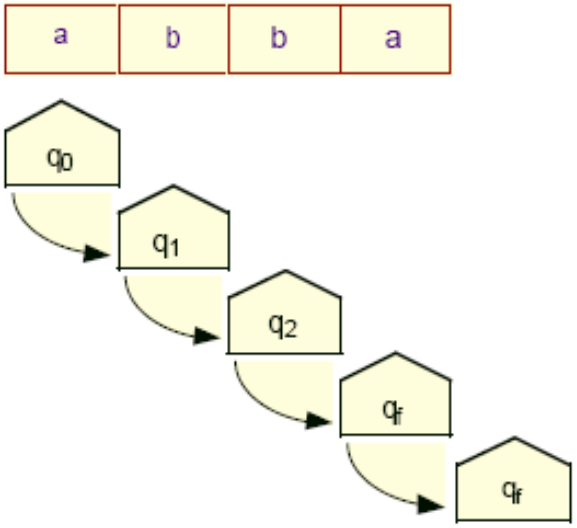
Qual á a Linguagem reconhecida/aceita por esse AF?

Exemplo:

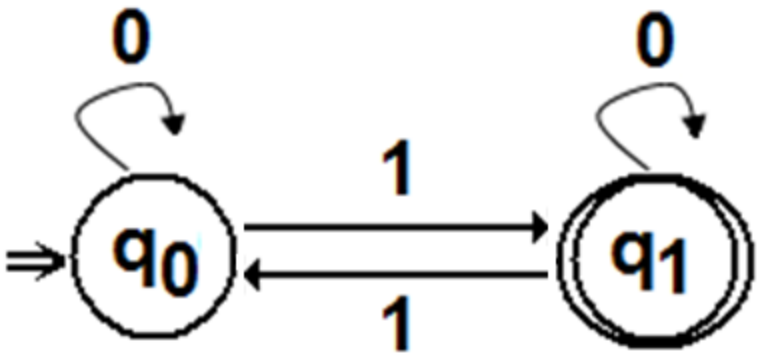


- algoritmo
  - q_1: o símbolo anterior é a
  - q_2: o símbolo anterior é b
  - após identificar aa ou bb: estado final q_f e varre o sufixo somente para terminar o processamento

- processamento de w = abba (aceita)

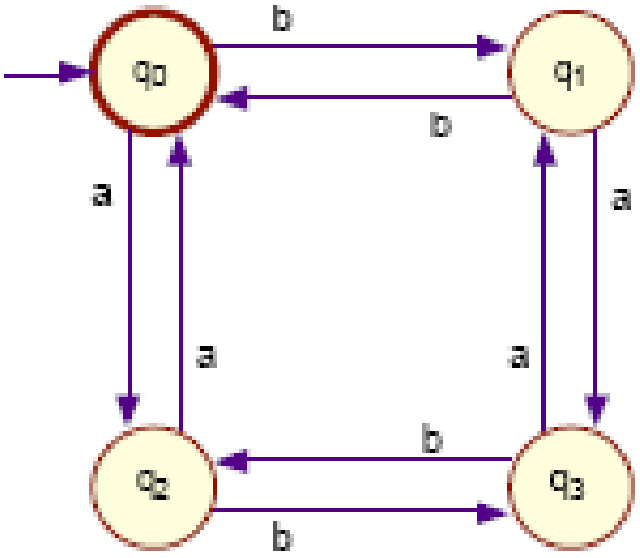


Exemplo:



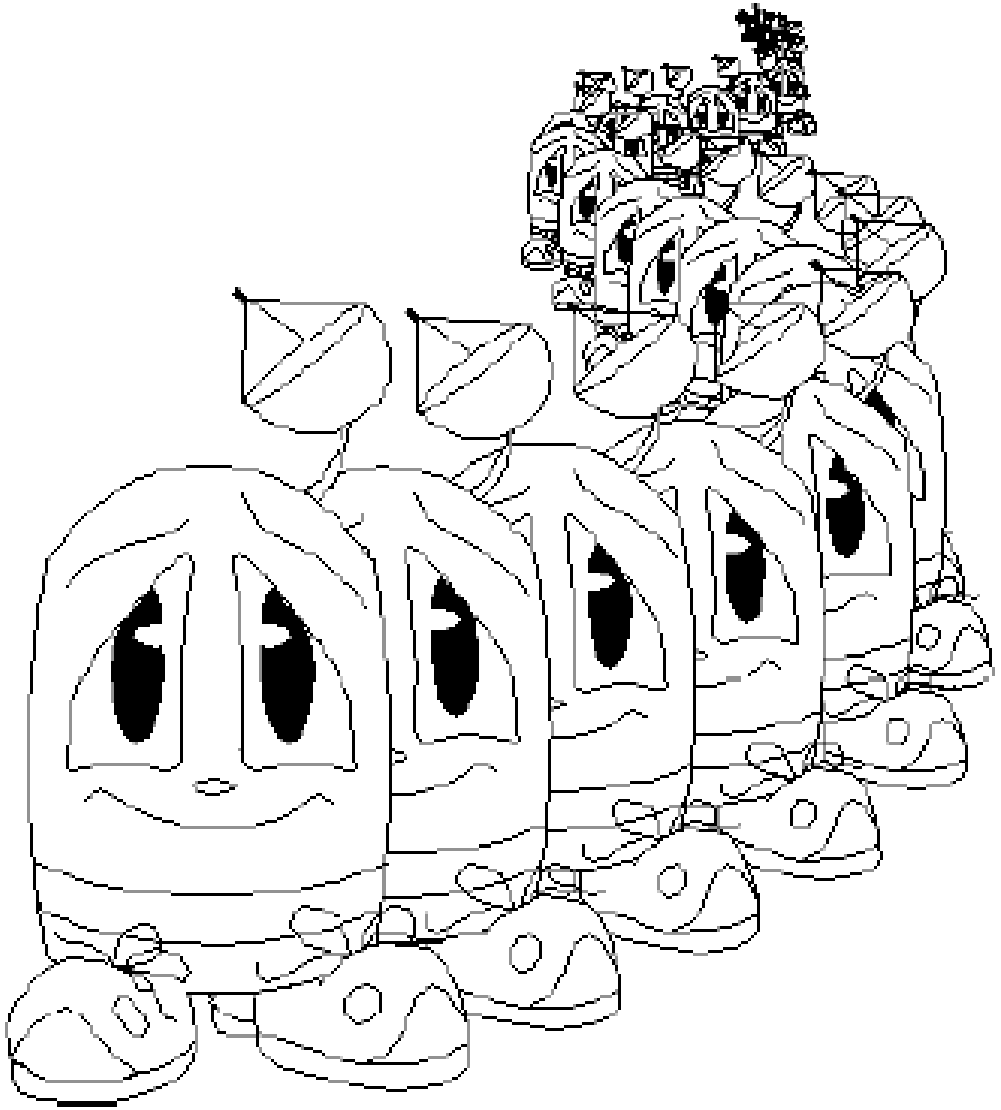
Qual á a Linguagem reconhecida/aceita por esse AF?

Exemplo:



Qual á a Linguagem reconhecida/aceita por esse AF?

Autômato Finito Não Determinístico (AFND)



Definição: Não Determinismo

É uma importante generalização dos modelos de máquinas abstratas, na Teoria da Computação.

Numa transição não determinista, partindo-se de um estado q_1, lendo-se um símbolo a, é possível ir para um ou mais estados. Por exemplo: $\delta(q_1, a) = \{q_1, q_2\}$.

Nem sempre o não-determinismo aumenta o poder de reconhecimento de uma classe de autômatos. Por exemplo, o Autômato Finito Não Determinístico tem poder de reconhecimento equivalente ao Autômato Finito Determinístico.

Definição: Autômato Finito Não Determinístico – Def1

Um AFND é um Autômato Finito, onde são permitidas transições não determinísticas.

Uma palavra é aceita no AFND se existe alguma seqüência de transições, processando a palavra símbolo a símbolo, que leve do estado inicial a um estado final.

Na representação de máquina abstrata de reconhecimento (fita de entrada + unidade de controle finito), o AFND assume um conjunto de estados alternativos, como se houvesse uma multiplicação de unidades de controle com processamento independente

Definição: Autômato Finito Não Determinístico – Def. 2

Um Autômato Finito é uma 5-upla:

$A = (Q, \Sigma, \delta, q_0, F)$

$Q \rightarrow$ conjunto finito de estados

$\Sigma \rightarrow$ alfabeto (símbolos da fita): $\Sigma \cap Q = \varnothing$

$q_0 \rightarrow$ estado inicial

$\delta \rightarrow$ função de transição de estado: $Q \times \Sigma \rightarrow 2^Q$,

onde 2^Q é o conjunto potência de Q (todos os subconjuntos)

$F \rightarrow$ conjuntos de estados finais (aceitação): $F \subseteq Q$

Definição: Função de Transição de Estados δ (AFND)

Para todos os estados do AFND ($q \in Q$), a função deve definir quais serão os possíveis estados do autômato ($\{q_1, q_{2, \ldots}\} \subset Q$), quando for lido qualquer símbolo na fita ($a \in \sum$).

A função δ é dada por uma lista de transições do tipo:

$$\delta(q, a) = \{q_1, q_2, \ldots, q_N\},$$

ou seja,

o AFND no estado q e lendo o símbolo a, escolhe um dos estados q_i ($1 \leq i \leq N$) como novo estado.

Exemplo 1:

δ_5	a	b
q_0	$\{q_0,q_1\}$	$\{q_0,q_2\}$
q_1	$\{q_f\}$	-
q_2	-	$\{q_f\}$
q_f	$\{q_f\}$	$\{q_f\}$

Qual á a Linguagem reconhecida/aceita por esse AF?

Exemplo 2:

- $M_6 = (\{a, b\}, \{q_0, q_1, q_2, q_f\}, \delta_6, q_0, \{q_f\})$

Qual á a Linguagem reconhecida/aceita por esse AF?

$$L_6 = \{w \mid w \text{ possui aaa como sufixo}\}$$

♦ Processamento

- união dos resultados da função programa aplicada a cada estado alternativo
- definição formal do comportamento
  - * necessário estender a função programa
  - * argumento: um conjunto finito de estados e uma palavra
- w ∈ ACEITA(M)
  - * pelo menos um caminho alternativo aceita w
- w ∈ REJEITA(M)
  - * todas as alternativas rejeitam w

Definição: Função de Transição Estendida (AFND)

Seja $A = (Q, \sum, \delta, q_0, F)$ um AFND.

A função δ estendida $\underline{\delta}: 2^Q \times \sum^* \rightarrow 2^Q$ estendida para palavra é assim definida:

$\underline{\delta}(q, \varepsilon) = \{q\}$

$\underline{\delta}(q, aw) = \underline{\delta}(\delta(q, a), w)$

$\underline{\delta}(S, w) = \cup_{q \epsilon S} \underline{\delta}(q, w)$

sendo:

$q \epsilon Q$ (estado)

$S \subset Q$ (subconjunto de estados)

$a \epsilon \sum$ (símbolo)

$w \epsilon \sum^*$ (palavra)

Definição: Palavra aceita por um AFND

Seja w ϵ $\sum^*$ e o AFND A = (Q, $\sum$, δ, q_0, F).
A palavra w é aceita por A se $\underline{\delta}(q_0, w) \cap F \neq \varnothing$,

ou seja:
se existe algum estado final em $\underline{\delta}(q_0, w)$.

Definição: Linguagem aceita por um AFND

A linguagem aceita por A é definida por:

$$L(A) = \{w \in \sum^* \mid \underline{\delta}(q_0, w) \cap F \neq \varnothing\}$$

♦ Determinismo × Não-Determinismo

- não-determinismo
  - aparentemente, um significativo acréscimo
  - ao poder computacional de um AF
- na realidade
  - não aumenta o poder computacional
- para cada AFN
  - é possível construir um AFD equivalente
  - (que realiza o mesmo processamento)
  - o contrário também é verdadeiro

Equivalência dentre AFD e AFND

Seja L(A) a linguagem aceita por um AFND A.
Existe um AFD A' que também aceita L(A).

Seja o AFND A = (Q, $\sum$, δ, q0, F).

O AFD A' equivalente a A é dado por A' = (Q', $\sum$, δ', q_0', F'), sendo:

- Q' = 2^Q. Elementos de Q' do tipo $\{q_1, q_2,...,q_k\}$, sendo q_i(i = 1, 2,...,k) ϵ Q, representados por <q1 q2 ... qk>
- F' → estados de Q' que contêm pelo menos 1 estado de F.
- Q_0' = $\{q_0\}$, representado por <q0>
- δ'(<q_1 q_2...q_j>, a) = <r_1 r_2...r_i> se e somente se
  δ($\{q_1, q_2, ... q_j\}$, a) = δ(q_1, a) ∪ δ (q_2, a) ∪ ... δ(q_j, a) = $\{r_1, r_2,...,r_i\}$,

Prova da equivalência dentre AFD e AFND

- **¿ AFD M' simula AFN M ?**
  - indução no tamanho da palavra
  - deve-se mostrar que

    $\delta'(\langle q_0\rangle, w) = \langle q_1 \ldots q_u\rangle$ sse $\delta(\{q_0\}, w) = \{q_1, \ldots, q_u\}$.
  - *Base da indução*. $|w| = 0$
    - $\delta'(\langle q_0\rangle, \varepsilon) = \langle q_0\rangle$ sse $\delta(\{q_0\}, \varepsilon) = \{q_0\}$
    - V por definição de função programa estendida
  - *Hipótese de indução*. $|w| = n, n \geq 1$
    - suponha que
    - $\delta'(\langle q_0\rangle, w) = \langle q_1 \ldots q_u\rangle$ sse $\delta(\{q_0\}, w) = \{q_1, \ldots, q_u\}$
  - *Passo de Indução*. $|wa| = n + 1$ e $n \geq 1$
    - $\delta'(\langle q_0\rangle, wa) = \langle p_1 \ldots p_v\rangle$ sse $\delta(\{q_0\}, wa) = \{p_1, \ldots, p_v\}$

    o que equivale, por hipótese de indução

    $\delta'(\langle q_1 \ldots q_u\rangle, a) = \langle p_1 \ldots p_v\rangle$ sse $\delta(\{q_1, \ldots, q_u\}, a) = \{p_1, \ldots, p_v\}$

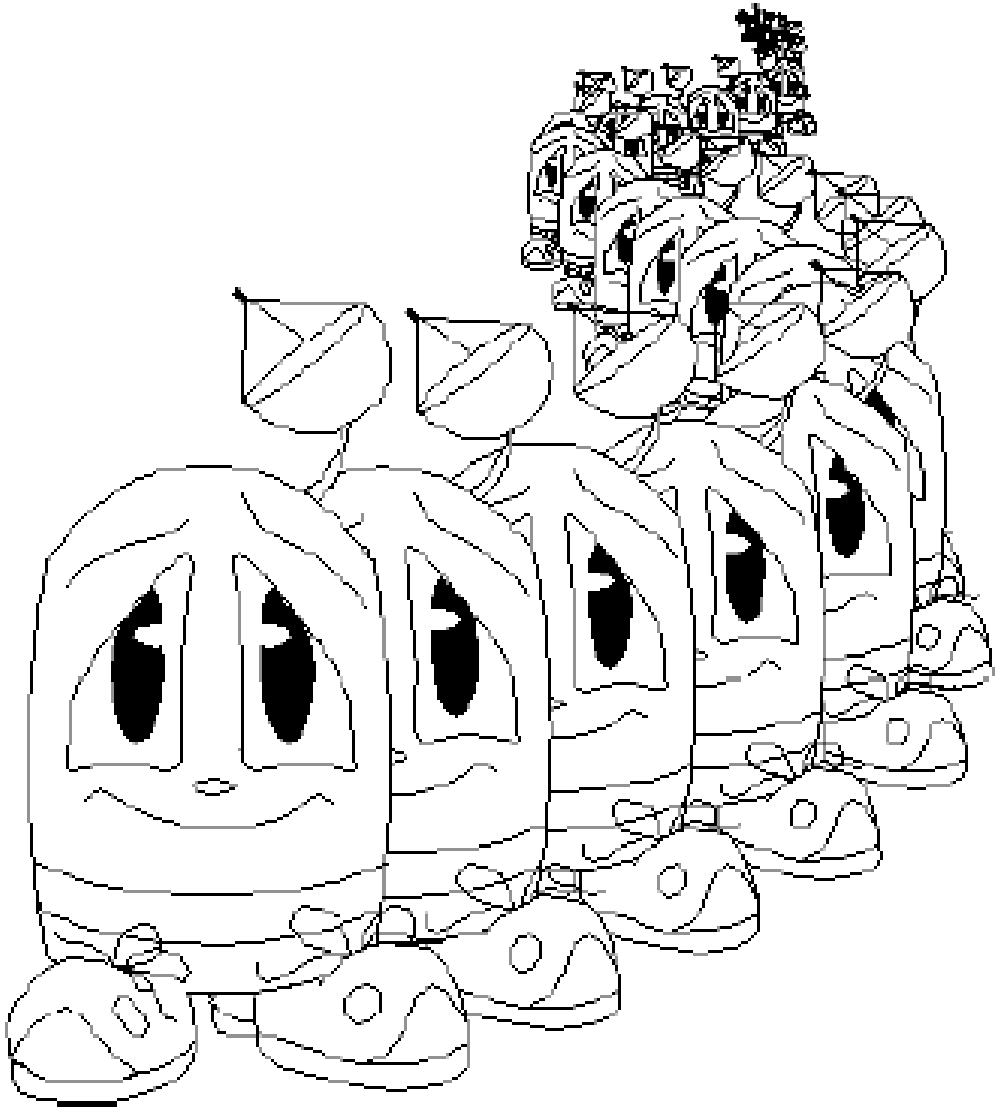
    o que é verdadeiro, por definição de δ'

- **Logo, M' simula M para qq w**

♦ *Exemplo:*

- AFN $M_6 = (\{a, b\}, \{q_0, q_1, q_2, q_f\}, \delta_6, q_0, \{q_f\})$
  - exemplo anterior
- AFD $M_6' = (\{a, b\}, Q', \delta_6', \langle q_0 \rangle, F')$
  - $Q' = \{\langle q_0 \rangle, \langle q_1 \rangle, \langle q_2 \rangle, \langle q_f \rangle, \langle q_0 q_1 \rangle, \langle q_0 q_2 \rangle, \ldots, \langle q_0 q_1 q_2 q_f \rangle\}$
  - $F' = \{\langle q_f \rangle, \langle q_0 q_f \rangle, \langle q_1 q_f \rangle, \ldots, \langle q_0 q_1 q_2 q_f \rangle\}$

Autômato Finito Não Determinístico com Movimentos Vazios (AFε)

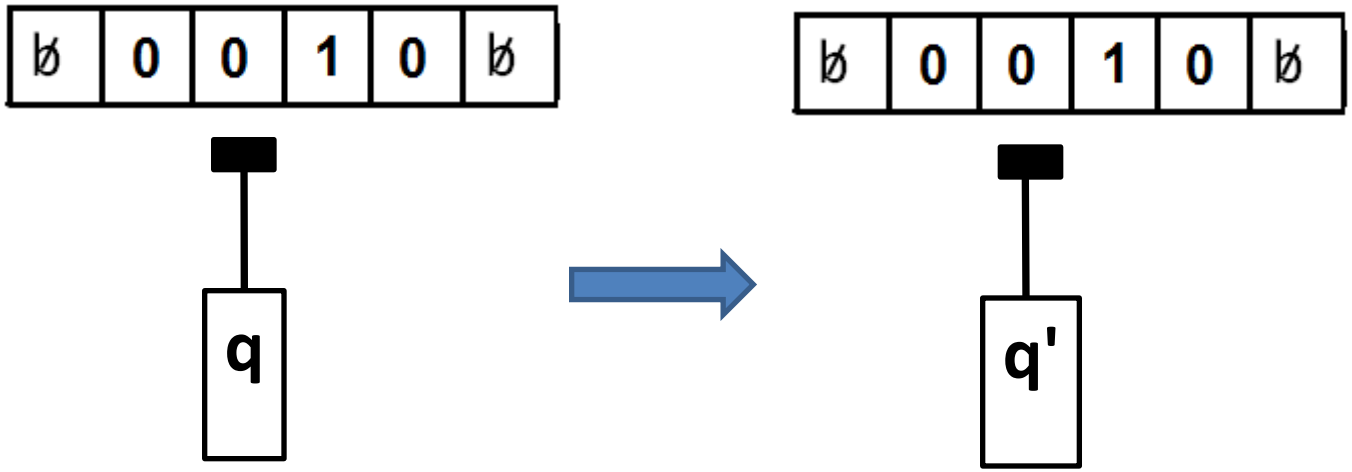


Definição: Transição Vazia (ou Movimento Vazio)

- É uma transição sem leitura na fita de entrada;

- Não aumenta o poder de reconhecimento dos AFs;

- Na visão de máquina reconhecedora, é como se o controle finito mudasse de estado sem mover a cabeça de leitura:



Definição: Autômato Finito Não Determinístico com Movimentos Vazios (AFε)

O AFND com transições vazias (AFε) é uma 5-upla $A = (Q, \sum, \delta, q_0, F)$, com Q, $\sum$, q_0 e F definidos como no AFND.

A função δ é dada por: $\delta: Q \times (\sum \cup \{\varepsilon\}) \rightarrow 2^Q$.

A função δ para transições vazias é do tipo:

$$\delta(q, \varepsilon) = \{q_1, q_2, \ldots, q_n\}$$

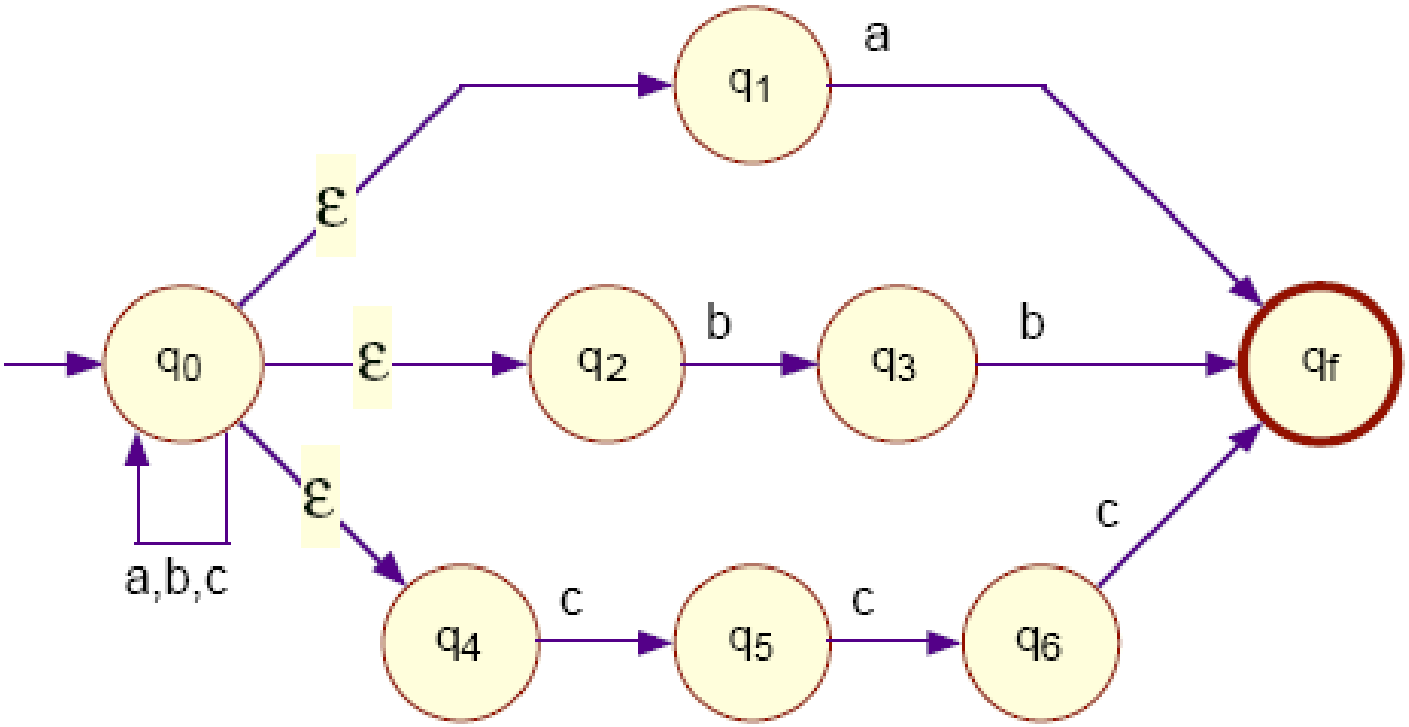
o AFε no estado q e sem ler nenhum símbolo na fita de entrada, escolhe qualquer um dos q_i ($i = 1, 2, \ldots, n$) para seu próximo estado.

A função δ para outras transições com leitura de símbolo é igual ao AFND: $\delta(q, a) = \{q_1, q_2, \ldots, q_n\}$

- *Exemplo:*

δ_7	a	b	ε
q_0	$\{q_0\}$	-	$\{q_f\}$
q_f	-	$\{q_f\}$	

- $M_8 = (\{a, b, c\}, \{q_0, q_1, q_2, q_3, q_4, q_5, q_6, q_f\}, \delta_8, q_0, \{q_f\})$



♦ Processamento (semântica)

- análogo ao de um AFN
- processamento de uma transição vazia
  - também é não-determinista
  - assume simultaneamente os estados destino e origem
  - origem de uma transição vazia sempre é um caminho alternativo

♦ Processamento (formal)

- função programa estendida
  - conjunto de estados
  - palavra
  - baseado na noção de fecho vazio

Definição: Função Fecho Vazio (F_ε)

Seja o AFε $A = (Q, \sum, \delta, q_0, F)$. A função $F\varepsilon: Q \rightarrow 2^Q$ é definida indutivamente:

$$F_\varepsilon(q) = \{q\}, \text{ se } \delta(q, \varepsilon) = \{\ \}$$

$$F_\varepsilon(q) = \{q\} \cup \delta(q, \varepsilon) \cup \left(\bigcup_{p \in \delta(q, \varepsilon)} F_\varepsilon(p) \right), \text{ caso contrário}$$

Informalmente:

Se q é um estado de Q, $F_\varepsilon(q)$ é o conjunto de estados que se pode atingir a partir de q, utilizando-se apenas transições ε, incluindo-se o próprio estado q.

Definição: Função Fecho Vazio Estendida($\underline{F}_\varepsilon$)

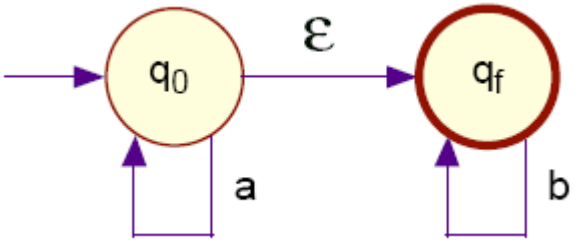
Seja o AFε A = (Q, $\sum$, δ, q_0, F), a função $\underline{F}_\varepsilon$: $2^Q \rightarrow 2^Q$ é assim definida:

$$\forall\ P \subset Q:\ \underline{F}_\varepsilon(P) = \bigcup_{q\ \epsilon\ P} F_\varepsilon(q)$$

Ou seja, se P = {q_1, $q_{2,\ \ldots}$}:

$$\underline{F}_\varepsilon(P) = F_\varepsilon(q_1) \cup F_\varepsilon(q_2) \cup \ldots$$

No exemplo:



$F_\varepsilon(q_0) = \{q_0, q_F\}$
$F_\varepsilon(q_F) = \{q_F\}$
$\underline{F}_\varepsilon(\{q_0, q_F\}) = \{q_0, q_F\}$

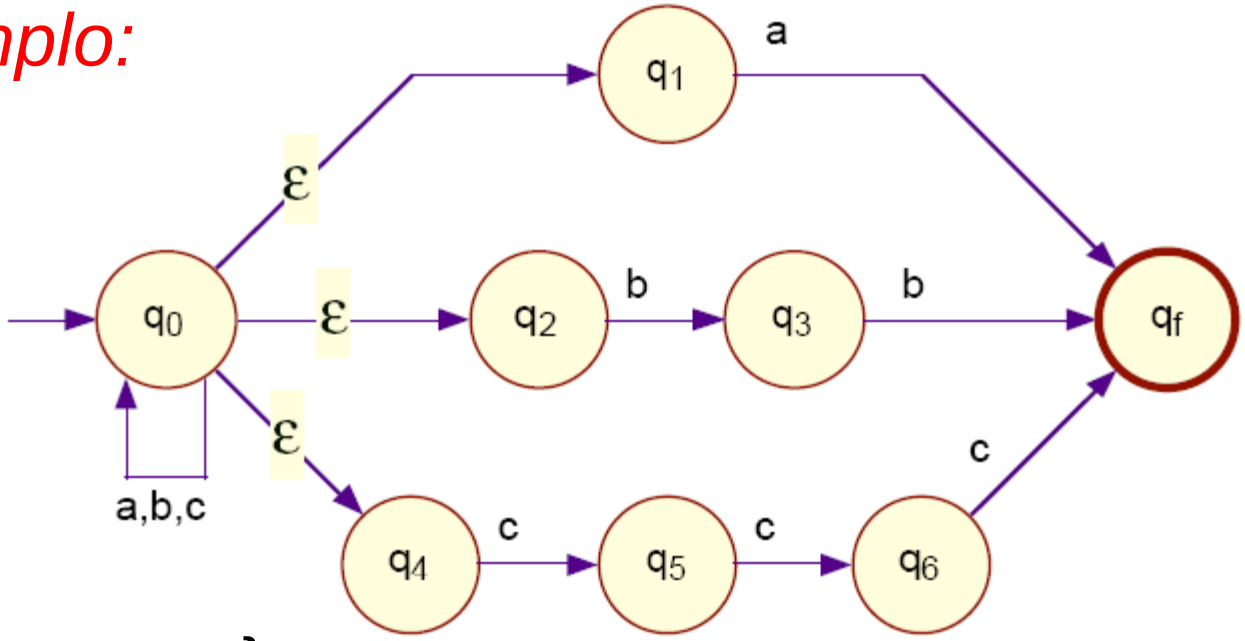
Definição: – Função Programa Estendida de um AFε ($\underline{\delta}$)

Seja o AFε A = $(Q, \sum, \delta, q_0, F)$. A função δ estendida denotada por $\underline{\delta}$: $2^Q \times \sum^* \rightarrow 2^Q$ estendida para um conjunto de estados e uma palavra e é dada por:

$$\underline{\delta}(P, \varepsilon) = \underline{F}_{\varepsilon}(P)$$

$$\underline{\delta}(P, wa) = \underline{F}_{\varepsilon}(R), \text{ onde } R = \{r \mid r \in \delta(s, a) \text{ e } s \in \underline{\delta}(P, w)\}$$

No exemplo:



$F_\varepsilon(q_0) = \{q_0, q_1, q_2, q_4\}$

$\underline{\delta}(\{q_0\}, abb) = F\varepsilon(\{r \mid r \in \delta(s, b) \text{ e } s \in \underline{\delta}(\{q_0\}, ab)\})$

$\underline{\delta}(\{q_0\}, ab) = F\varepsilon(\{r \mid r \in \delta(s, b) \text{ e } s \in \underline{\delta}(\{q_0\}, a)\})$

$\underline{\delta}(\{q_0\}, a) = F\varepsilon(\{r \mid r \in \delta(s, a) \text{ e } s \in \underline{\delta}(\{q_0\}, \varepsilon)\})$

* $\underline{\delta}(\{q_0\}, \varepsilon) = F\varepsilon(\{q_0\}) = \{q_0, q_1, q_2, q_4\}$
* $\underline{\delta}(\{q_0\}, a) = \{q_0, q_1, q_2, q_4, q_f\}$
* $\underline{\delta}(\{q_0\}, ab) = \{q_0, q_1, q_2, q_3, q_4\}$
* $\underline{\delta}(\{q_0\}, abb) = \{q_0, q_1, q_2, q_4, q_f\}$

Equivalência entre AFND e AFε

- uma linguagem é regular sse é aceita por um AFε
- a capacidade de reconhecimento dos AFε é a mesma dos AFD e dos AFN
- seja $M = (\Sigma, Q, \delta, q_0, F)$ um AFε qualquer
- seja $M' = (\Sigma, Q, \delta', q_0, F')$ um AFN
- δ'
  - $\delta' : Q \times \Sigma \rightarrow 2^Q$
  - $\delta'(q, a) = \underline{\delta}(\{q\}, a)$
- F'
  - conjunto de todos $q \in Q$ tq
  - algum elemento do $F\varepsilon(q)$ pertence a F

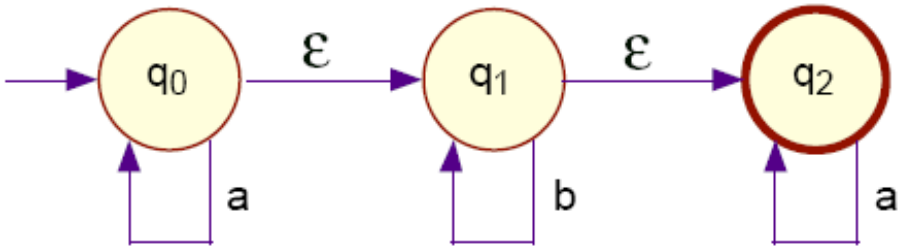
Prova da equivalência dentre AFND e AFε

¿ AFN simula o AFε ?

* indução no tamanho da palavra
* exercício

- $\delta_9'(q_0, a) = \underline{\delta}_9(\{q_0\}, a) =$
  $F\varepsilon(\{r \mid r \in \delta(s, a) \text{ e } s \in \underline{\delta}(\{q_0\}, \varepsilon)\}) = \{q_0, q_1, q_2\}$
- $\delta_9'(q_0, b) = \underline{\delta}_9(\{q_0\}, b) =$
  $F\varepsilon(\{r \mid r \in \delta(s, b) \text{ e } s \in \underline{\delta}(\{q_0\}, \varepsilon)\}) = \{q_1, q_2\}$
- $\delta_9'(q_1, a) = \underline{\delta}_9(\{q_1\}, a) =$
  $F\varepsilon(\{r \mid r \in \delta(s, a) \text{ e } s \in \underline{\delta}(\{q_1\}, \varepsilon)\}) = \{q_2\}$
- $\delta_9'(q_1, b) = \underline{\delta}_9(\{q_1\}, b) =$
  $F\varepsilon(\{r \mid r \in \delta(s, b) \text{ e } s \in \underline{\delta}(\{q_1\}, \varepsilon)\}) = \{q_1, q_2\}$
- $\delta_9'(q_2, a) = \underline{\delta}_9(\{q_2\}, a) =$
  $F\varepsilon(\{r \mid r \in \delta(s, a) \text{ e } s \in \underline{\delta}(\{q_2\}, \varepsilon)\}) = \{q_2\}$
- $\delta_9'(q_2, b) = \underline{\delta}_9(\{q_2\}, b) =$
  $F\varepsilon(\{r \mid r \in \delta(s, b) \text{ e } s \in \underline{\delta}(\{q_2\}, \varepsilon)\})$ é indefinida

Exemplo:



- $M_9' = (\{a, b\}, \{q_0, q_1, q_2\}, \delta_9', q_0, F')$

AFND resultante:

Expressão Regular (ER)

aa

ba^*

$(a + b)^*$

$(a + b)^*aa(a + b)^*$

$a^*ba^*ba^*$

$(a + b)^*(aa + bb)$

$(a + \varepsilon)(b + ba)^*$

Expressão Regular

- toda LR pode ser descrita por uma
  - *Expressão Regular*
- formalismo
  - simples
  - denotacional (gerador)
- definida a partir de
  - conjuntos (linguagens) básicos
  - operação de concatenação
  - operação de união
- são adequadas para a comunicação
  - homem × homem
  - homem × máquina

Definição: Expressão Regular

- sobre um alfabeto Σ
- indutivamente definida
  - $\emptyset$ é ER e denota a *linguagem vazia*
  - ε é ER e denota a linguagem $\{\varepsilon\}$
  - **x** é ER onde **x** $\in \Sigma$ e denota a linguagem $\{x\}$
  - se **r** e **s** são ER e denotam as linguagens R e S, então

(r+s)	é ER e denota $R \cup S$
(rs)	é ER e denota RS
(r*)	é ER e denota R^*

Expressão Regular: Precedência entre Operadores

- concatenação sucessiva tem precedência sobre
  - concatenação
  - união
- concatenação tem precedência sobre
  - união

Expressão Regular: Precedência entre Operadores

- concatenação sucessiva tem precedência sobre
  - * concatenação
  - * união
- concatenação tem precedência sobre
  - * união

♦ Linguagem Gerada

- por uma ER r
- é representada por L(r) ou GERA(r)

Exemplos

ER	Linguagem Representada
aa	
ba^*	
$(a + b)^*$	
$(a + b)^*aa(a + b)^*$	
$a^*ba^*ba^*$	
$(a + b)^*(aa + bb)$	
$(a + \varepsilon)(b + ba)^*$	

- **As ER denotam exatamente as LR**

- ***Teorema***

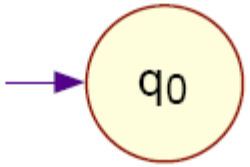
  - Se r é uma ER,
  - então GERA(r) é uma LR

- **Prova**

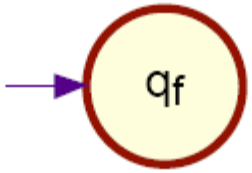
  - L é LR sse é possível construir um
    - AF (AFD, AFN ou AFε), que reconheça L
  - portanto, é necessário mostrar que,
    - dado uma ER r qq,
    - é possível construir um AF M tq
    - ACEITA(M) = GERA(r)
  - demonstração de que ACEITA(M) = GERA(r)
    - indução no número de operadores
  - é assumido que qq AF
    - pode ser simulado por um AF com exatamente um estado final

♦ Base (ER com zero operadores)

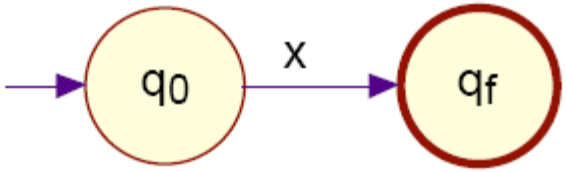
- se r tem zero operadores, então é da forma:
  - * $r = \emptyset$
  - * $r = \varepsilon$
  - * $r = x \quad (x \in \Sigma)$
- $r = \emptyset$. $M_1 = (\emptyset, \{q_0\}, \delta_1, q_0, \emptyset)$



- $r = \varepsilon$. $M_2 = (\emptyset, \{q_f\}, \delta_2, q_f, \{q_f\})$



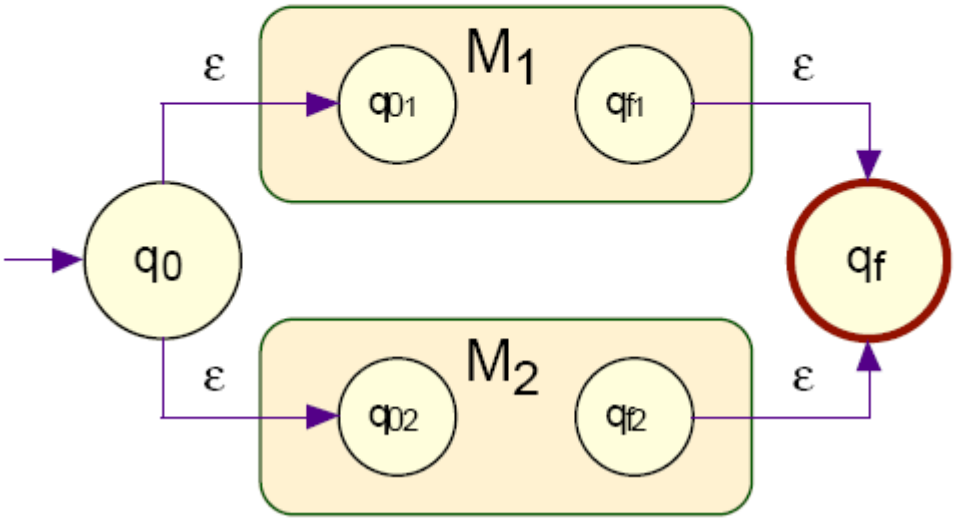
- **r = x**. $M_3 = (\{x\}, \{q_0, q_f\}, \delta_3, q_0, \{q_f\})$



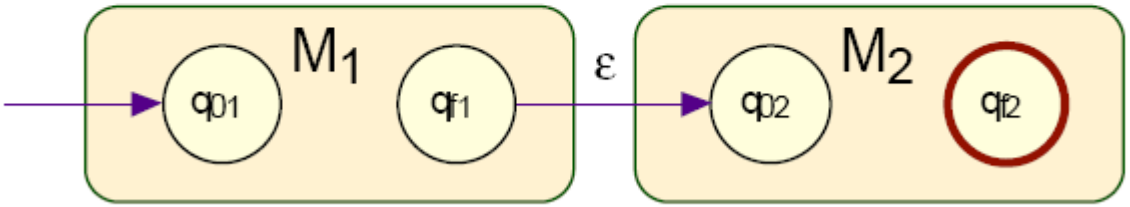
- **Hipótese (ER com até $n > 0$ operadores)**
  - suponha que é possível construir um autômato finito que aceita a linguagem Gera(r)

- **Indução (ER com $n + 1$ operadores)**
  - se r possui $n + 1$ operadores, então a ER pode ser representada por (r_1 e r_2 possuem conjuntamente no máximo n operadores)
    - $r = r_1 + r_2$
    - $r = r_1 r_2$
    - $r = r_1^*$
  - por hipótese de indução existem
    - $M_1 = (\Sigma_1, Q_1, \delta_1, q_{0_1}, \{q_{f_1}\})$ tq $L(M_1) = GERA(r_1)$
    - $M_2 = (\Sigma_2, Q_2, \delta_2, q_{0_2}, \{q_{f_2}\})$ tq $L(M_2) = GERA(r_2)$

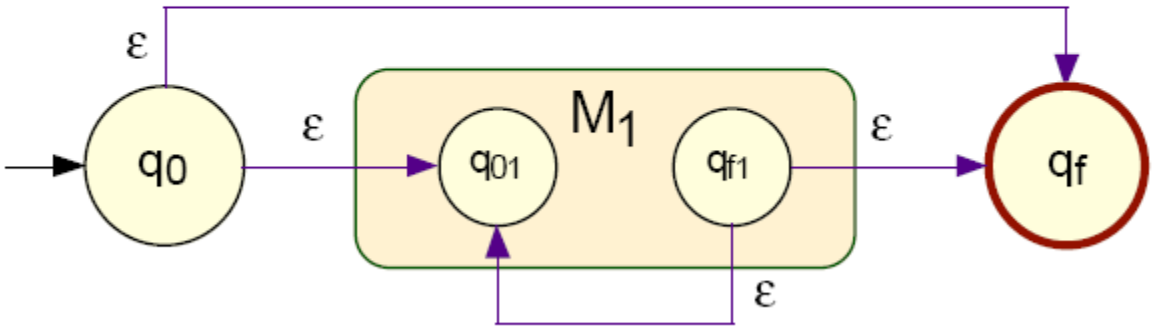
- $r = r_1 + r_2$
  - $M = (\Sigma_1 \cup \Sigma_2, Q_1 \cup Q_2 \cup \{q_0, q_f\}, \delta, q_0, \{q_f\})$



- **$r = r_1 r_2$**
  - $M = (\Sigma_1 \cup \Sigma_2, Q_1 \cup Q_2, \delta, q_{0_1}, \{q_{f_2}\})$

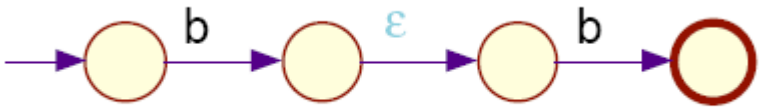


- $r = r_1^*$
  - $M = (\Sigma_1, Q_1 \cup \{q_0, q_f\}, \delta, q_0, \{q_f\})$

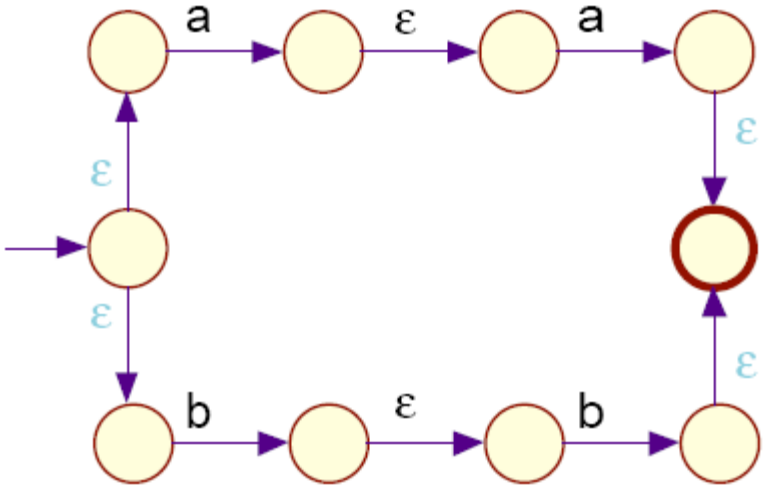


Exemplos

bb

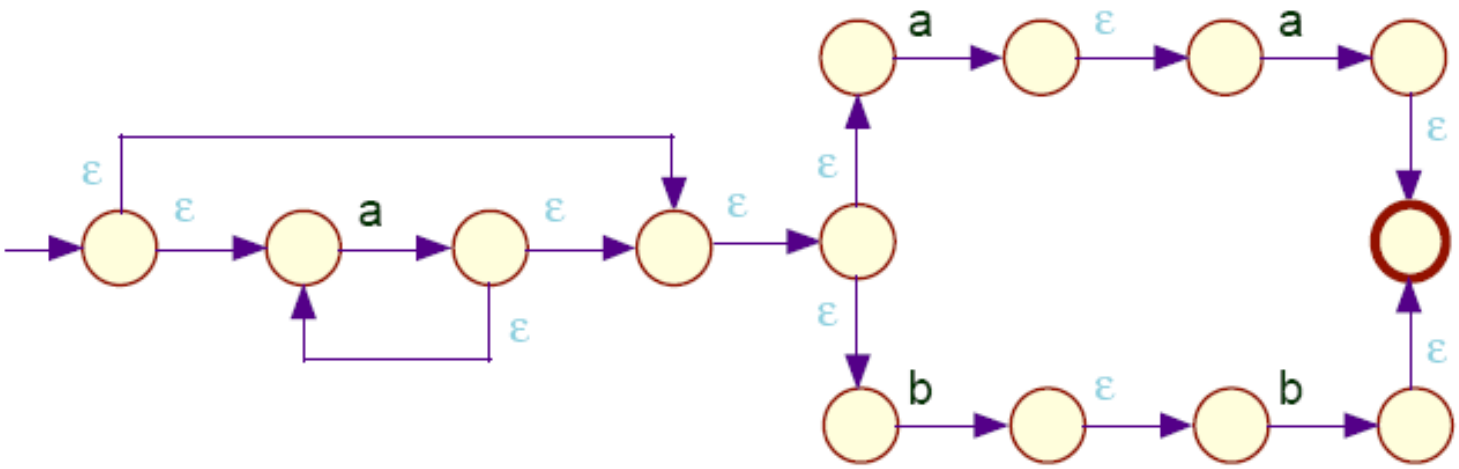


(aa + bb)



Exemplos

$a^*(aa + bb)$



Conversão AFD => ER

Transparência Extra

Gramática Regular

G = (V, T, P, S), sendo:

V → Conjunto finito de variáveis (não-terminais).
T → Conjunto finito de símbolos terminais.
S → Símbolo não terminal inicial (S ϵ V).
P → Conjunto finito de regras $\alpha \rightarrow \beta$, sendo $\alpha \in (V \cup T)^+$ e $\beta \in (V \cup T)^*$

G = (V, T, P, S) é Regular, se toda produção em P é da forma:

$A \rightarrow wB$ ou $A \rightarrow Bw$ ou $A \rightarrow w$ com $A \in V$, $B \in V$ e $w \in T^*$, ou seja:

Uma variável à esquerda e no máximo uma variável à direita (a variável a direita não pode ser precedida e sucedida ao mesmo tempo por terminais).

Gramática Linear

- Seja $G = (V, T, P, S)$ uma gramática e
  - sejam $A, B \in V$ (variáveis)
  - $w \in T^*$ (palavra de símbolos terminais)

  Então G é

- ***Gramática Linear à Direita (GLD)***
  - $A \rightarrow wB$ ou
  - $A \rightarrow w$

- ***Gramática Linear à Esquerda (GLE)***
  - $A \rightarrow Bw$ ou
  - $A \rightarrow w$

- ***Gramática Linear Unitária à Direita (GLUD)***
  - como na linear à direita
  - adicionalmente, $|w| \leq 1$

- ***Gramática Linear Unitária à Esquerda (GLUE)***
  - como na linear à esquerda
  - adicionalmente, $|w| \leq 1$

- ***Exemplo.*** **a(ba)***
  - **GLD**. G = ({S, A}, {a, b}, P, S) onde P é tq
    - S → aA
    - A → baA | ε
  - **GLE**. G = ({S}, {a, b}, P, S) onde P é tq
    - S → Sba | a
  - **GLUD**. G = ({S, A, B}, {a, b}, P, S) onde P é tq
    - S → aA
    - A → bB | ε
    - B → aA
  - **GLUE**. G = ({S, A}, {a, b}, P, S) onde P é tq
    - S → Aa | a
    - A → Sb

Equivalência Gramática Regular (GLUD) e AF (AFε)

- suponha $G = (V, T, P, S)$ uma GLUD
- seja AFε $M = (\Re, Q, \delta, q_0, F)$ tq
  - $\Re = T$
  - $Q = V \cup \{q_f\}$
  - $F = \{q_f\}$
  - $q_0 = S$
  - δ é como segue

Tipo da Produção	Transição Gerada
$A \rightarrow \varepsilon$	$\delta(A, \varepsilon) = q_f$
$A \rightarrow a$	$\delta(A, a) = q_f$
$A \rightarrow B$	$\delta(A, \varepsilon) = B$
$A \rightarrow aB$	$\delta(A, a) = B$

- *Exemplo*
  - $G = (\{S, A, B\}, \{a, b\}, P, S)$
    - $S \rightarrow aA$
    - $A \rightarrow bB \mid \varepsilon$
    - $B \rightarrow aA$
  - AFε $M = (\{a, b\}, \{S, A, B, q_f\}, \delta, S, \{q_f\})$ onde δ é tq

Prova

- ***Teorema.***
  - Se L é gerada por uma GR,
  - então L é uma LR
- **Prova**
  - mostrar que
    - dado uma GR G qq,
    - é possível construir um AF M tq
    - ACEITA(M) = GERA(G)
  - M simula as derivações de G
  - demonstração de que ACEITA(M) = GERA(r)
    - indução no número de derivações
  - indução no número de derivações
  - suponha $\alpha \in (T \cup V)^*$
  - suponha $w \in T^*$

- **Base: $S \Rightarrow^1 \alpha$**
  - $\alpha = \varepsilon$ Existe $S \rightarrow \varepsilon$ Logo, $\delta(S, \varepsilon) = q_f$
  - $\alpha = a$ Existe $S \rightarrow a$ Logo, $\delta(S, a) = q_f$
  - $\alpha = A$ Existe $S \rightarrow A$ Logo, $\delta(S, \varepsilon) = A$
  - $\alpha = aA$ Existe $S \rightarrow aA$ Logo, $\delta(S, a) = A$
- **Hipótese: $S \Rightarrow^n \alpha$**
  - $\alpha = w$ Suponha que $\underline{\delta}(S, w) = q_f$
  - $\alpha = wA$ Suponha que $\underline{\delta}(S, w) = A$

- **Indução: $S \Rightarrow^{n+1} \alpha$**
  - então, $S \Rightarrow^{n} wA \Rightarrow^{1} \alpha$
    - $\delta(S, w) = A$ é a única hipótese que importa
  - $\alpha = w\varepsilon = w$. Existe $A \rightarrow \varepsilon$. Logo,

    $\delta(S, w\varepsilon) = \delta(\delta(S, w), \varepsilon) = \delta(A, \varepsilon) = q_f$
  - $\alpha = wb$. Existe $A \rightarrow b$. Logo,

    $\delta(S, wb) = \delta(\delta(S, w), b) = \delta(A, b) = q_f$
  - $\alpha = wB$. Existe $A \rightarrow B$. Logo,

    $\delta(S, w\varepsilon) = \delta(\delta(S, w), \varepsilon) = \delta(A, \varepsilon) = B$
  - $\alpha = wbB$. Existe $A \rightarrow bB$. Logo,

    $\delta(S, wb) = \delta(\delta(S, w), b) = \delta(A, b) = B$

Equivalência AF (AFD) e Gramática Regular (GLUD)

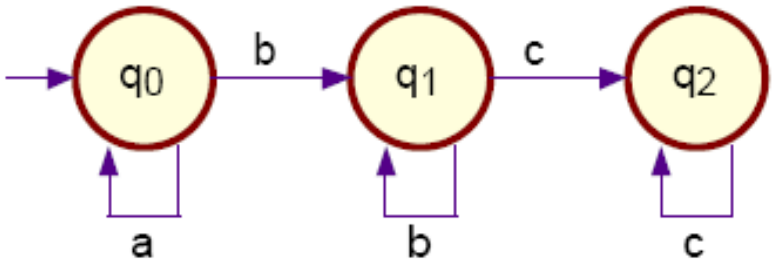
- suponha AFD $M = (\Sigma, Q, \delta, q_0, F)$ tq ACEITA(M) =
- seja $G = (V, T, P, S)$ uma GLUD tq
  - $V = Q \cup \{S\}$
  - $T = \Re$
  - P é tq (suponha $q_i, q_k \in Q$, $a \in \Re$ e $q_f \in F$)

Transição	Produção
-	$S \rightarrow q_0$
-	$q_f \rightarrow \varepsilon$
$\delta(q_i, a) = q_k$	$q_i \rightarrow aq_k$

- G = ({q_0, q_1, q_2, S}, {a, b, c}, S, P) onde P é tq

Exemplo:

- AFD M = ({a, b, c}, {q_0, q_1, q_2}, δ, q_0, {q_0, q_1, q_2})



- ***Teorema:*** **Se L é uma LR, então existe G, GR que gera L**
  - dado um AFD M qq,
    - construção GR G
    - tq GERA(G) = ACEITA(M)
  - construção de uma GLUD
    - derivação simula função programa estendida
  - demonstração de que AGERA(G) = ACEITA(M)
    - indução no tamanho da palavra
    - indução no tamanho da palavra
    - suponha w elemento de $\Re^*$

Prova

- **Base $|w| = 0$**
  - por definição
    - $S \rightarrow q_0$ é produção
  - se $w = \varepsilon \in$ ACEITA(M), então
    - q_0 é estado final
    - $q_0 \rightarrow \varepsilon$ é produção
    - $S \Rightarrow q_0 \Rightarrow \varepsilon$
- **Hipótese $|w| = n$ e $n > 1$**
  - seja w tal que $\underline{\delta}(q_0, w) = q$
    - q *não* é final, então suponha $S \Rightarrow wq$
    - q *é* final, então suponha $S \Rightarrow wq \Rightarrow w$

Prova (cont.)

- **Indução $|wa| = n + 1$**
  - seja w tal que $\underline{\delta}(q_0, wa) = p$
  - então, $\delta(\underline{\delta}(q_0, w), a) = \delta(q, a) = p$
    - $S \Rightarrow wq$ é a única hipótese que importa
  - Portanto, se:
    - p *não* é estado final

      então $S \Rightarrow^* wq \Rightarrow^1 wap$
    - p *é* estado final

      então $S \Rightarrow^* wq \Rightarrow^1 wap \Rightarrow^1 wa$

Minimização de Autômatos Finitos Determinísticos

- **Objetivo**
  - gerar um AF equivalente
  - com o menor número de estados possível
- **O autômato mínimo é único**
  - a minimização de AF distintos
    - que aceitam a mesma linguagem
    - geram o mesmo AF mínimo
- **Idéia básica do algoritmo**
  - unificar os *estados equivalentes*

♦ *Definição. Estados Equivalentes*

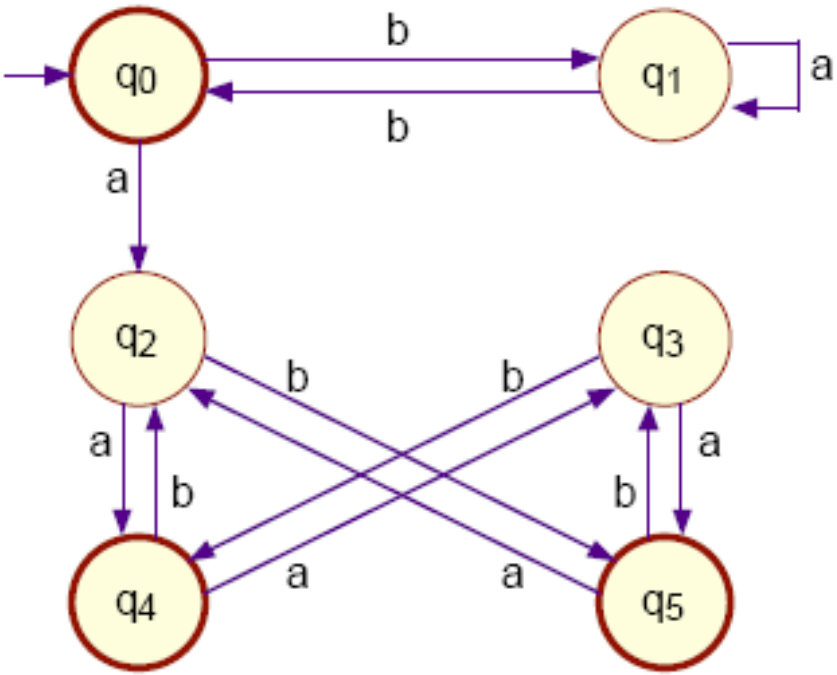
- q e p são equivalentes sse
  - * para qq w,
  - * $\delta(q, w)$ e $\delta(p, w)$
  - * resultam simultaneamente em estados finais, ou não-finais
- ou seja
  - * processamento de uma entrada qq
  - * a partir de estados equivalentes
  - * gera o mesmo resultado aceita/rejeita

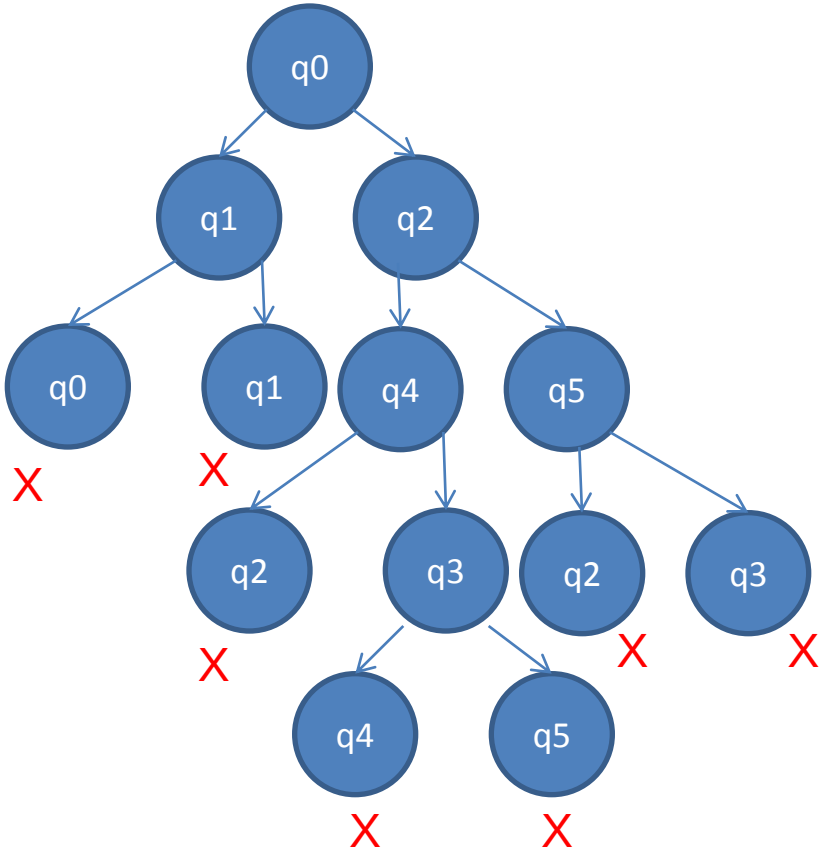
- ***Pré-Requisitos do Algoritmo***
  - AF deve ser determinístico
  - não pode ter estados inacessíveis
    - não-atingíveis a partir do estado inicial
  - a função programa deve ser total
- **Caso o AF não satisfaça algum dos pré-requisitos**
  - gerar um AFD equivalente
    - algoritmos introduzidos nos teoremas
  - eliminar os estados inacessíveis e suas correspondentes transições
  - função programa total
    - introduzir um novo estado não-final d
    - incluir as transições não-previstas, tendo como resultado o estado d
    - incluir um ciclo em d para todos os símbolos do alfabeto

Algoritmo de Minimização pelo Particionamento da Tabela de Transição

Transparência Extra

Exemplo:





Estados Acessíveis = {q0, q1, q2, q3, q4, q5}
Conclusão: nenhum estado é eliminado

δ	a	b
qo	q2	q1
q1	q1	qo
q2	q4	q5
q3	q5	q4
q4	q3	q2
q5	q2	q3

1a Partição: Finais e Não finais

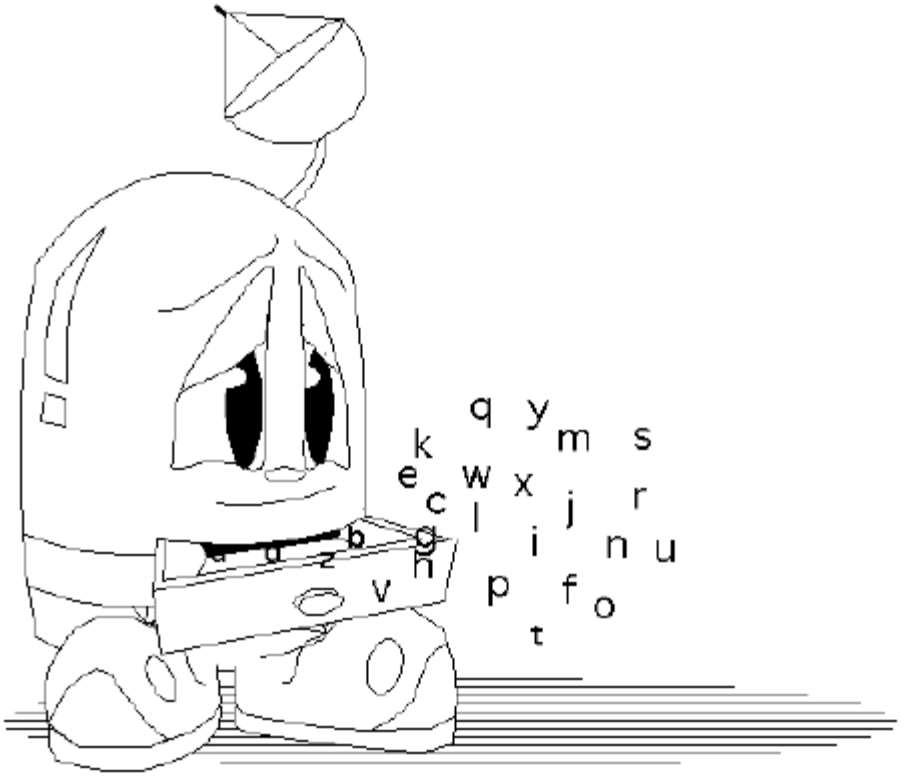
δ	a	b	π_0		a	b
qo	q2	q1	q1	I		
q1	q1	qo	q2			
q2	q4	q5	q3			
q3	q5	q4	q0	II		
q4	q3	q2	q4			
q5	q2	q3	q5			

Algoritmo de Minimização pelo Particionamento da Tabela de Estados

(Livro Paulo B. Menezes)

- **Idéia básica do algoritmo**
  - identifica os estados equivalentes
    - por *exclusão*
  - a partir de uma tabela de estados
    - são marcados os estados não-equivalentes
  - ao final do algoritmo
    - as referências não-marcadas
    - representam os estados equivalentes.

Autômato Finito com Saída: Máquina de Moore e Máquina de Mealy



♦ Geração de uma palavra de saída

- estende a definição de AF
- *não* altera a capacidade de reconhecimento
  - reconhece a classe de liguagens regulares
- saída
  - *não* pode ser lida
  - *não* pode ser usada como memória auxiliar
- as saídas podem ser associadas
  - às ***transições*** - Máquina de **Mealy**
  - aos ***estados*** - Máquina de **Moore**

Saída

- definida sobre um alfabeto especial
  - *alfabeto de saída*
  - pode ser igual ao alfabeto de entrada
- saída
  - fita independente da de entrada
- cabeça da fita de saída
  - move uma célula para direita
  - a cada símbolo gravado
- resultado do processamento
  - estado final (condição de aceita/rejeita)
  - informação contida na fita de saída

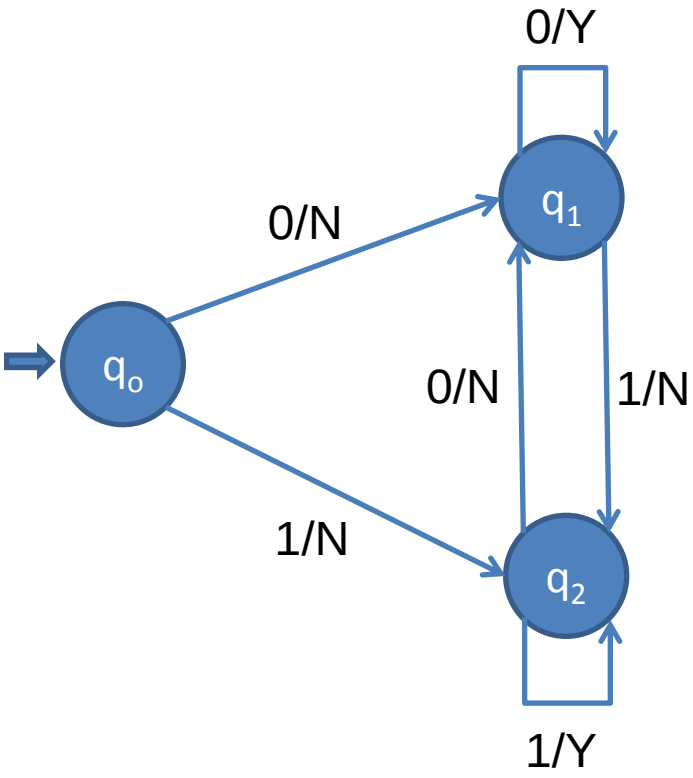
Máquina de Mealy

- para cada transição
  - gera uma palavra de saída
  - pode ser vazia

♦ *Definição. Máquina de Mealy*

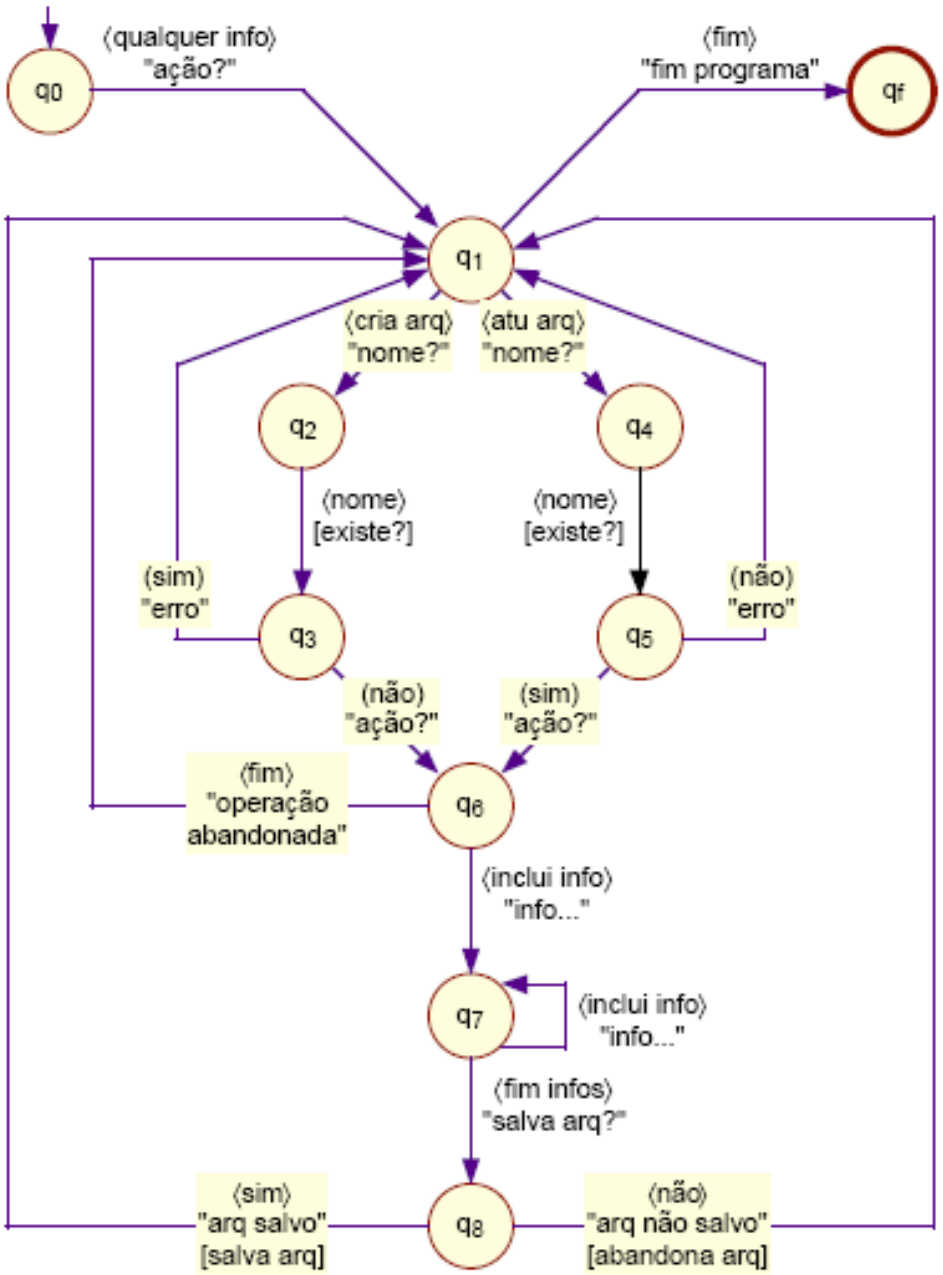
- 6-upla $M = (\Sigma, Q, \delta, q_0, F, \Delta)$
- Σ alfabeto de símbolos de entrada
- Q conjunto finito de estados
- δ função programa ou de transição
  - $\delta: Q \times \Sigma \rightarrow Q \times \Delta^*$
  - função parcial
- q_0 estado inicial tq $q_0 \in Q$
- F conjunto de estados finais tq $F \subseteq Q$
- Δ alfabeto de símbolos de saída

Exemplo 1:



Exemplo 2:

- ***Exemplo*: programa × usuário**
  - cria e atualiza arquivos
  - ⟨...⟩ entrada fornecida pelo usuário
    - exemplo: teclado
  - "..." saída gerada pelo programa
    - exemplo: vídeo
  - [...] ação interna ao programa
    - sem comunicação com o usuário
  - (...) resultado de uma ação interna
    - entrada no diagrama
  - $M = (\Sigma, \{q_0, q_1, \ldots, q_8, q_f\}, \delta, q_0, \{q_f\}, \Delta)$
    - $\Sigma = \Delta$
    - símbolos válidos no diálogo



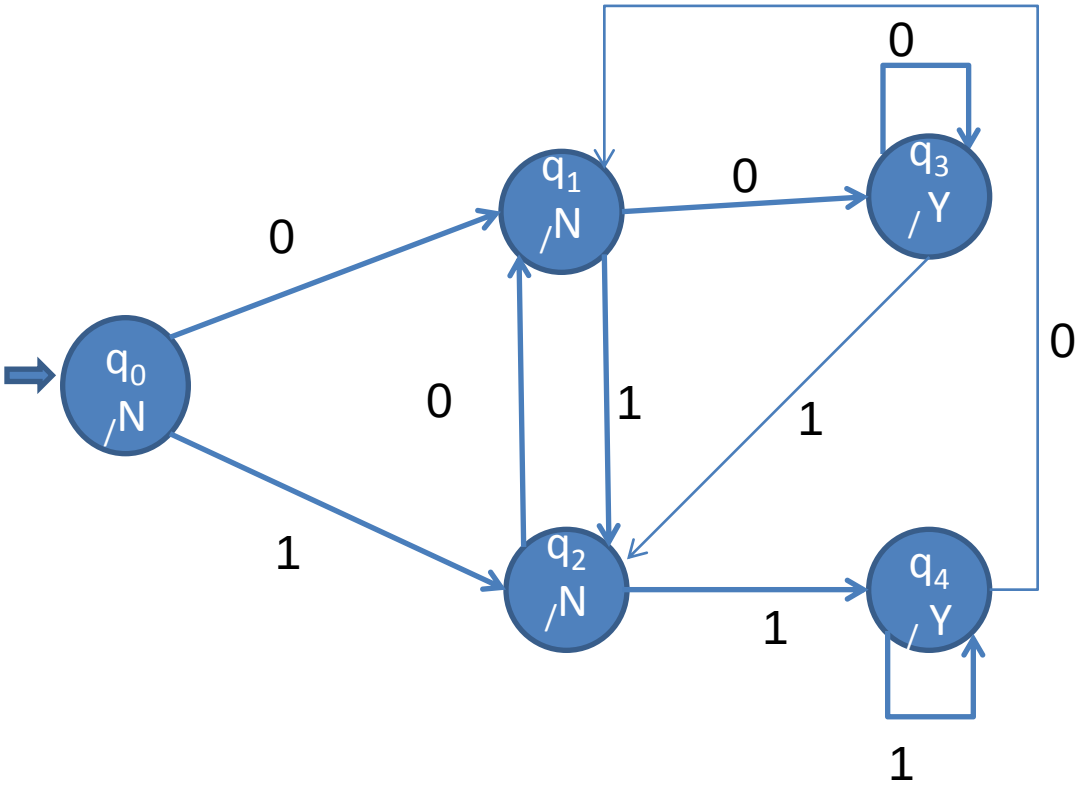
Máquina de Moore

- para cada estado da máquina
  - gera uma palavra de saída
  - pode ser vazia
- possui uma função de saída

♦ *Máquina de Moore*

- 7-upla $M = (\Sigma, Q, \delta, q_0, F, \Delta, \delta_S)$
- Σ alfabeto de símbolos de entrada
- δ função programa ou de transição
  - $\delta: Q \times \Sigma \rightarrow Q$
  - função parcial
- Q conjunto finito de estados
- q_0 estado inicial tq $q_0 \in Q$
- F conjunto de estados finais tq $F \subseteq Q$
- Δ alfabeto de símbolos de saída
- δ_S função de saída
  - $\delta_S: Q \rightarrow \Delta^*$
  - *é* total

Exemplo:



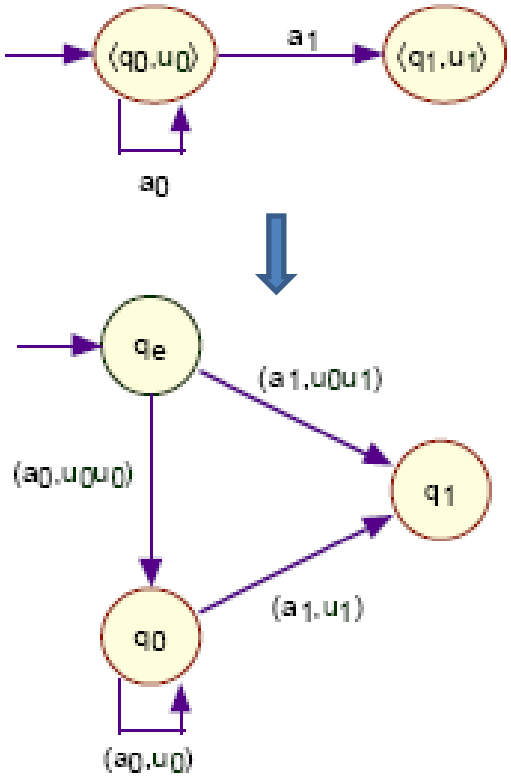
♦ *Exemplo*: Analisadores Léxicos

- p/ compiladores ou tradutores de linguagens
- analisador léxico
  - basicamente é um AF
  - em geral, determinístico
- identifica os componentes básicos da linguagem
  - números
  - identificadores
  - separadores
  - etc
- *estado final*
  - associado a cada unidade léxica identificada
  - *saída*: descreve ou codifica a unidade léxica
- *estado não-final*
  - *saída*: palavra vazia
- como seria uma correspondente Máquina de Mealy?

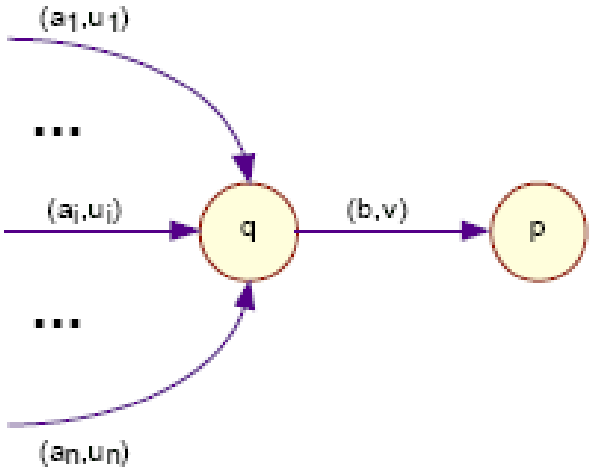
Equivalência Moore × Mealy

- **Equivalência**
  - *não* é válida para a entrada vazia
  - Máquina de Moore
    - gera a palavra correspondente ao estado inicial
  - Máquina de Mealy
    - não gera saída
    - não executa transição alguma
  - demais casos
    - *equivalência*

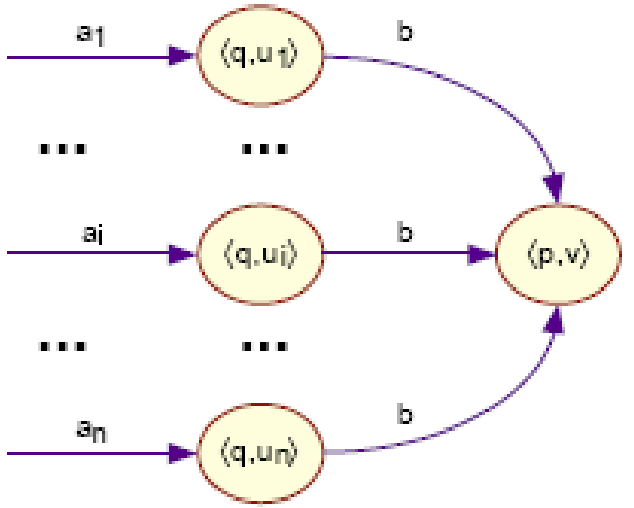
- ***Teorema*: Moore → Mealy**
  - qq Máq de Moore
  - pode ser simulada por uma Máq de Mealy
  - para *entradas não-vazias*
- **Correspondente Máq de Mealy**
  - simulação da saída do estado inicial de Moore?
    - introduz um novo estado q_e
    - referenciado somente na primeira transição a ser executada
    - saída referente ao estado inicial de Moore



- ***Teorema*. Mealy → Moore**
  - Qq Máquina de Mealy
  - pode ser simulada por uma Máquina de Moore

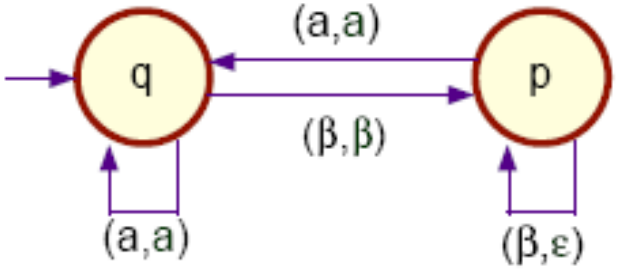


- **Correspondente Máq de Moore**
  - transições com saídas diferentes que atingem um mesmo estado
    - um estado para cada símbolo de saída
    - em geral, aumenta o número de estados

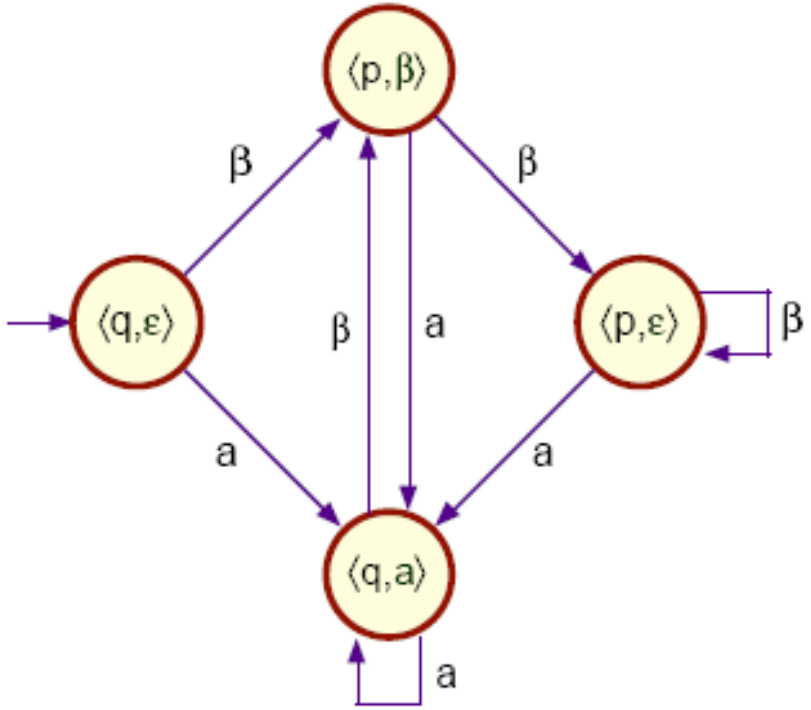


♦ *Exemplo*: Compactação de brancos

- compacta os brancos de um texto
- Máq. Mealy
  - * ME = ({a, β}, {q, p}, δ_{ME}, q, {q, p}, {a, β})
  - * a representa um símbolo qualquer
  - * β o símbolo branco
  - * δ_{ME} - na etiqueta de uma transição
    primeira componente: símbolo lido
    segunda componente: palavra gravada



- Máquina de Moore
  - * MO = ({a, β}, Q, δ_{MO}, ⟨q, ε⟩, F, {a, β}, δ_S)
  - * Q = F = {q, p}×{ε, a, β}
  - * δ_{MO} e δ_S onde, em cada estado, a segunda componente representa a saída

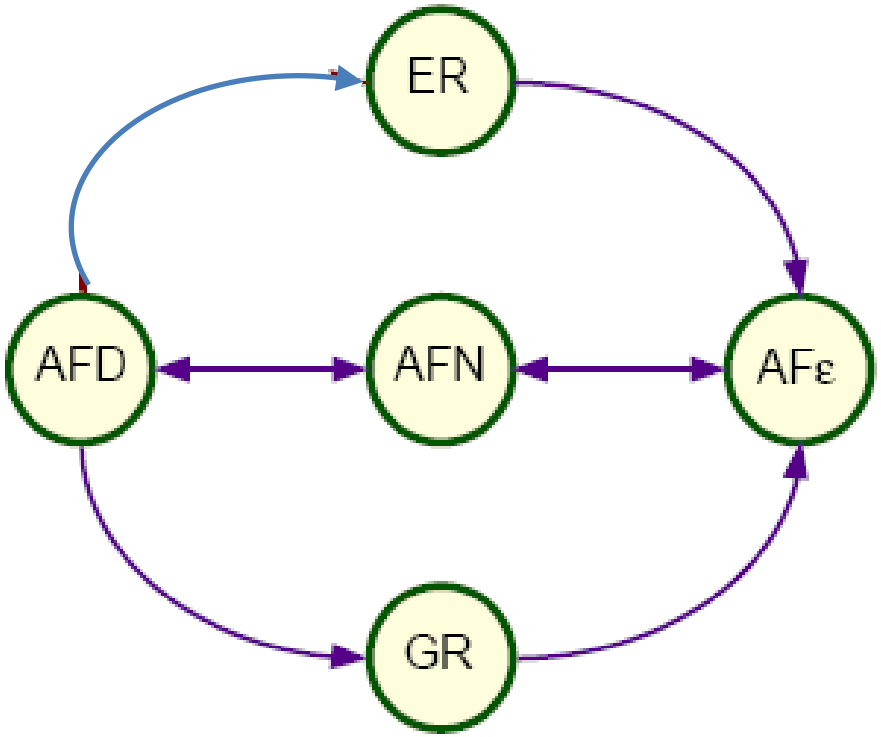


Propriedades das Linguagens Regulares

- **LR podem ser representadas por formalismos**
  - baixa complexidade
  - grande eficiência
  - fácil implementação
- **Entretanto, é uma classe de linguagens**
  - restrita
  - limitada
  - é fácil definir linguagens que *não* são regulares

♦ A análise de cada propriedade

- é desenvolvida para *um* formalismo
- demais formalismos
  - * é suficiente traduzir



Questões sobre LR

- como determinar se uma
  - linguagem *é regular*?
- como verificar se uma LR é
  - *infinita*?
  - *finita*?
  - *vazia*?
- é possível analisar duas LR e concluir se são
  - *iguais*?
  - *diferentes*?
- ou seja, é possível verificar
  - se duas especificações são equivalentes?
  - se dois compiladores são equivalentes?
  - se o compilador aceita exatamente a linguagem especificada?
- a classe das LR é fechada para operações de
  - *união*?
  - *concatenação*?
  - *intersecção*?

Operações Fechadas sobre LR

- **Obs**
  - para uma linguagem L sobre Σ^*,
    L' denota o seu complemento em Σ^*
- ***Teorema*: A classe das LR é fechada para**
  - união
  - concatenação
  - complemento
  - intersecção

♦ Prova

- *união* e *concatenação*
  - * decorrem trivialmente da definição de ER
- *complemento*
  - * a idéia consiste em inverter as condições de ACEITA/REJEITA do AF
- inversão das condições ACEITA/REJEITA
  - * modifica o AF, garantindo que somente irá parar ao terminar de ler toda a entrada
  - * transforma os estados finais em não-finais e vice-versa
- *intersecção*
  - * tratar a intersecção em termos da união e complemento

♦ Complemento

- seja L uma LR sobre Σ^*
- seja $M = (\Sigma, Q, F, q_0, \delta)$ um AFD tq ACEITA(M) = L
- seja $M' = (\Sigma, Q', F', q_0, \delta')$ tq ACEITA(M') = L'
  - $Q' = Q \cup \{d\}$
  - $F' = Q' - F$
  - δ' é como δ, excetuando-se

    $\delta'(q, a) = d$ se $\delta(q, a)$ não é definida

    $\delta'(d, a) = d$

♦ Intersecção

- Sejam L_1 e L_2 LR
- $L_1 \cap L_2 = (L_1' \cup L_2')'$
- como a classe das LR é fechada para
  - * complemento
  - * união

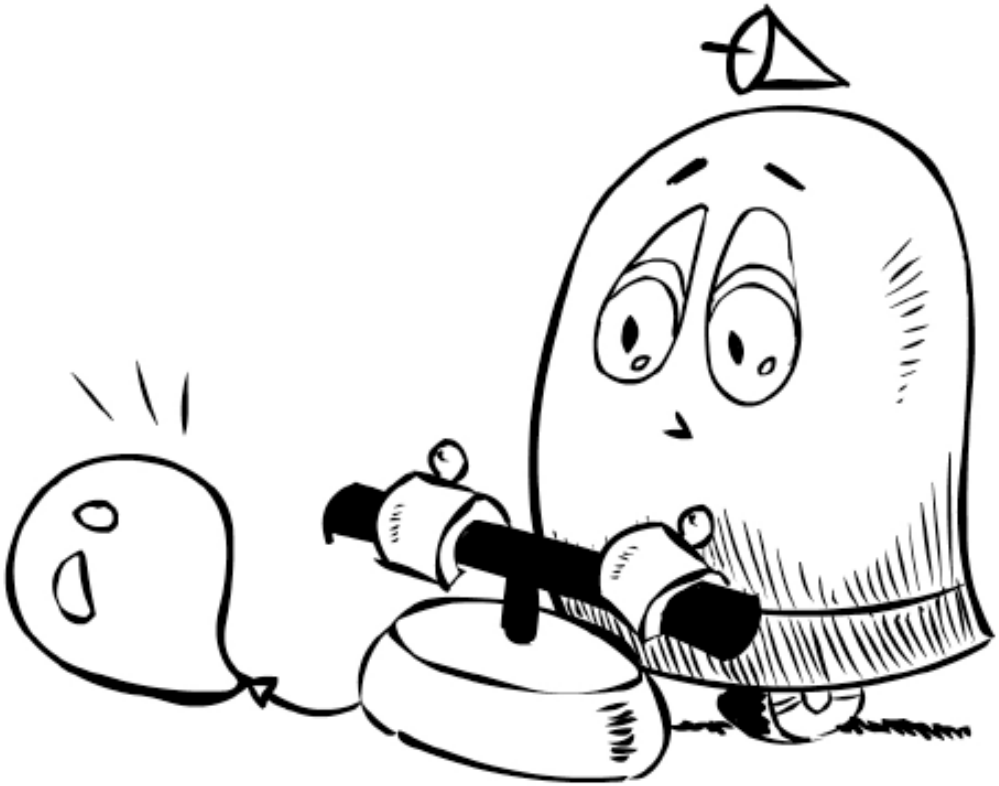
  então é fechada para a intersecção

- *Vazia* sse *não* aceita qualquer w tq $|w| < n$
  - processa M
    - para todo w tq $|w| < n$
  - se rejeita todas as palavras
    - a linguagem é *vazia*
  - prova
    - simples usando o bombeamento
    - exercício
- *Finita* sse *não* aceita w tq $n \leq |uv| < 2n$
  - conseqüência direta do caso infinita
    - negação do caso *infinita*
  - processa M
    - para todo w tq $n \leq |w| < 2n$
  - se rejeita todas as palavras
    - a linguagem é *finita*

Igualdade de Linguagens Regulares

- ***Teorema.***
  - se M_1 e M_2 são AF
  - então existe algoritmo para determinar se
    - $ACEITA(M_1) = ACEITA(M_2)$
- **Portanto**
  - existe um algoritmo para verificar se
    - dois AF são equivalentes
    - reconhecem a mesma linguagem
- **Prova**
  - sejam M_1 e M_2 AF tq
    - $ACEITA(M_1) = L_1$ e $ACEITA(M_2) = L_2$
  - seja $L_3 = (L_1 \cap L_2') \cup (L_1' \cap L_2)$
  - portanto, é possível construir um AF M_3 tq
    - $ACEITA(M_3) = L_3 = (L_1 \cap L_2') \cup (L_1' \cap L_2)$
  - claramente, $L_1 = L_2$ sse L_3 é *vazia*
    - existe algoritmo para verificar se LR é vazia
    - existe algoritmo pra $\cup$, $\cap$ e complemento

Lema do Bombeamento para LR



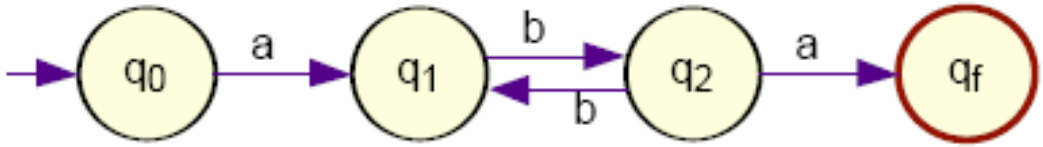
♦ Bombeamento para as LR

- proposição útil no estudo das propriedades
- LR
  - * é aceita por um AFD com n estados
- AFD reconhece w tq $|w| \geq n$
  - * obrigatoriamente assume algum estado q mais de uma vez
  - * existe um ciclo no programa que passa por q
- logo, w pode ser dividida em três subpalavras
  - * $w = uvz$
  - * $|uv| \leq n$ e $|v| \geq 1$
  - * v é a parte reconhecida pelo ciclo
- claramente uv^iz para qq $i \geq 0$
  - * é aceita pelo AFD
  - * executando o ciclo i vezes

- ***Teorema*: Se L é LR, então**
  - existe n tq,
  - para qq $w \in L$ onde $|w| \geq n$,
  - w pode ser definida como $w = uvz$
    - $|uv| \leq n$ e $|v| \geq 1$
  - para todo $i \geq 0$, $uv^iz \in L$

- **Prova**
  - L é LR
    - existe AFD $M = (\Sigma, Q, \delta, q_0, F)$ tq $L(M) = L$
  - seja n o cardinal de Q
  - seja $w = a_1a_2...a_m \in L$ tq $m \geq n$
  - suponha $\delta(q_0,a_1) = q_1$, $\delta(q_1,a_2) = q_2,...,\delta(q_{m-1},a_m) = q_m$
  - como $m \geq n$ existem r, s onde $0 \leq r < s \leq n$ tq
    - $q_r = q_s$
    - $\underline{\delta}(q_0, a_1...a_r) = q_r$, $\underline{\delta}(q_r, a_{r+1}...a_s) = q_s$
    - $\underline{\delta}(q_s, a_{s+1}...a_m) = q_m$
  - sejam $u = a_1...a_r$, $v = a_{r+1}...a_s$ e $z = a_{s+1}...a_m$
  - como $r < s \leq n$, então $|v| \geq 1$ e $|uv| \leq n$
  - como $q_r = q_s$, então
    - v é reconhecida em um ciclo
    - portanto, $uv^iz \in L$, para todo $i \geq 0$

♦ Exemplo. Bombeamento das LR



- n = 4
- no caso particular de w = abbba
  - $q_r = q_s = q_1$
  - u = a
  - v = bb
  - z = ba

- ***Exemplo*: Não é LR**

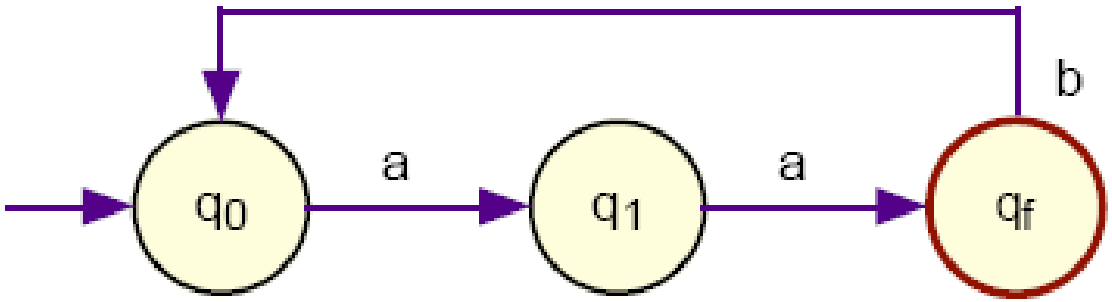
$L = \{w \mid w$ possui o mesmo número de símbolos a e b$\}$

- **Prova**
  - bombeamento + absurdo
  - suponha que L é LR
  - então existe AFD M com n estados que aceita L
  - seja $w = a^n b^n$
  - pelo bombeamento
    - $w = uvz$ onde $|uv| \leq n$, $|v| \geq 1$
    - para todo $i \geq 0$, $uv^i z$ é palavra de L
  - ABSURDO!!!
    - $|uv| \leq n$ e uv é composta exclusivamente por símbolos a
    - então, por exemplo $uv^2 z$, *não* pertence a L (não possui o mesmo número de símbolos a e b)

Investigação se LR é Vazia, Finita ou Infinita

- ***Teorema*: Se L é LR aceita por um AF M com n estados, então L é:**
  - **vazia** sse M *não* aceita qualquer w tq $|w| < n$
  - **finita** sse M *não* aceita w tq $n \leq |w| < 2n$
  - **infinita** sse M aceita w tq $n \leq |w| < 2n$
- **Ou seja, prova-se que**
  - *existe um algoritmo* para verificar se uma LR representada por um AF é *vazia*, *finita* ou *infinita*
- **Pelo bombeamento**
  - se aceita w tq $|w| \geq n$
  - então a linguagem é infinta
  - mas como verificar se existe tal w?

♦ *Exemplo*. LR Infinita



- infinita sse aceita w tq $n \leq |w| < 2n$
- menor palavra aceita de comprimento maior ou igual a 3
  - * aabaa
  - * possui comprimento 5
  - * ou seja, $3 \leq |aabaa| < 6$

- *Aceita* w tq $n \leq |w| < 2n$, então é *Infinita*
  - seja $w \in L$ tq $n \leq |w| < 2n$
  - bombeamento
    - $w = uvz$ onde $|uv| \leq n$, $|v| \geq 1$
    - para todo $i \geq 0$, uv^iz é palavra de L
  - logo, L é *infinita*

- *Infinita*, então *aceita* w tq $n \leq |w| < 2n$
  - se é infinita, então existe w tq $|w| \geq n$
  - se $|w| < 2n$, então a prova esta completa
  - suponha que *não* existe w tq $|w| < 2n$
  - suponha $|w| \geq 2n$
    - mas de comprimento menor ou igual a qualquer outra t tq $|t| \geq 2n$
  - então $w = uvz$ onde $|uv| \leq n$, $|v| \geq 1$ e $uv^iz \in L$
  - em, particular, $1 \leq |v| \leq n$ e $uz \in L$
  - ABSURDO!!!, pois relativamente a uz tem-se que:
  - se $|uz| \geq 2n$
    - w *não* é a menor palavra tq $|w| \geq 2n$!!!
  - se $|uz| < 2n$
    - como $|uvz| \geq 2n$ e $1 \leq |v| \leq n$
    - então $n \leq |uz| < 2n$
    - logo existe $w = uz$ tq $n \leq |w| < 2n$